



UNIVERSITÀ DEGLI STUDI DI UDINE

DIPARTIMENTO DI SCIENZE MATEMATICA, INFORMATICHE E FISICHE

TESI MAGISTRALE IN INFORMATICA

ALGORITMI E RAGIONAMENTO AUTOMATICO

STIMA DELLA CARDINALITÀ NEI FLUSSI DI DATI DISTRIBUITI:

MERGE E COMPATIBILITÀ SEMANTICA DEGLI ALGORITMI DI SKETCH

RELATORE

PROF. GABRIELE PUPPIS

LAUREANDO MAGISTRALE

DANIELE FERROLI

MATRICOLA

137357

ANNO ACCADEMICO

2024-2025

“I LEAVE SISYPHUS AT THE FOOT OF THE MOUNTAIN.
ONE ALWAYS FINDS ONE’S BURDEN AGAIN.
BUT SISYPHUS TEACHES THE HIGHER FIDELITY
THAT NEGATES THE GODS AND RAISES ROCKS.
HE TOO CONCLUDES THAT ALL IS WELL.
THIS UNIVERSE HENCEFORTH WITHOUT A MASTER
SEEMS TO HIM NEITHER STERILE NOR FUTILE.
EACH ATOM OF THAT STONE, EACH MINERAL
FLAKE OF THAT NIGHT-FILLED MOUNTAIN, IN ITSELF,
FORMS A WORLD. THE STRUGGLE ITSELF TOWARD
THE HEIGHTS IS ENOUGH TO FILL A MAN’S HEART.
ONE MUST IMAGINE SISYPHUS HAPPY.” — ALBERT CAMUS

Ringraziamenti

Questo percorso universitario, iniziato nel 2017, giunge ora a una conclusione dopo quasi nove anni.

Ho iniziato il mio percorso accademico con la laurea triennale in Informatica all'Università di Udine nel 2017. Allora avevo diciannove anni, come la maggior parte dei miei compagni di corso; oggi, invece, ne ho ventotto. Tra di loro c'erano anche alcuni miei amici e compagni di classe provenienti dalla mia scuola superiore. Oltre a loro, durante il primo anno ho fatto nuove amicizie e, in particolare, ho conosciuto Cristina.

I primi tre anni sono trascorsi molto rapidamente e sono stato circondato da amicizie importanti, universitarie e non. Verso il terzo anno è scoppiata una pandemia globale, che ha reso lo studio e la frequenza universitaria particolarmente difficili. Quando non era ancora terminata, mi sono ritrovato a dover scegliere un percorso magistrale senza avere davvero il tempo di capire con certezza cosa volessi fare.

Dopo tre anni di studio universitario mi sentivo già stanco e incerto su cosa fare. Vedevo i miei compagni andare avanti e proseguire il loro percorso, mentre io mi sentivo di essere rimasto fermo.

Due anni fa ho deciso di iniziare a lavorare, anche se mi mancavano ancora tre esami universitari.

A posteriori, cambierei alcune scelte fatte, ma il passato ormai è passato. Restano però le conseguenze delle proprie azioni. Fortunatamente però, tra queste, ci sono i legami che ho scelto di coltivare nel tempo: le amicizie, la mia relazione con Cristina e il rapporto con la mia famiglia.

Dedico questa tesi a chi mi ha sostenuto e a chi continua a sopportarmi ogni giorno. In particolare, ringrazio il mio relatore Gabriele Puppis per avermi aiutato nella realizzazione di questa tesi. La dedico anche alle persone che non ci sono più, ma che mi hanno cresciuto, accompagnato nella vita e fatto conoscere il mondo.

Grazie.

Indice

RINGRAZIAMENTI	v
NOTAZIONI E CONCETTI PRELIMINARI	i
1 INTRODUZIONE	5
1.1 Struttura della tesi	6
2 FONDAMENTI TEORICI	7
2.1 Modello di streaming	7
2.2 Algoritmi di streaming	9
2.3 Funzioni di hash	10
2.4 Sketch: strutture dati riassuntive	11
2.5 Famiglia di algoritmi per il conteggio degli elementi distinti	13
2.6 Altri obiettivi delle strutture riassuntive	14
2.7 Stimatori	14
2.8 Metriche empiriche di valutazione	16
3 ANALISI DELLO STATO DELL'ARTE	19
3.1 Obiettivo del problema e vincoli teorici	19
3.2 Sviluppo storico	20
3.2.1 Probabilistic Counting with Stochastic Averaging	21
3.2.2 LogLog	23
3.2.3 HyperLogLog	25
3.2.4 HyperLogLog++	27
3.3 Merge di algoritmi di sketch e scenari distribuiti	29
3.4 Confronto sintetico tra gli approcci	30
3.5 Altri problemi di sketching	31
3.5.1 Bloom filter	31
3.5.2 Count-Min Sketch	32
3.6 Related works	33
4 IMPLEMENTAZIONE	35
4.1 Obiettivi progettuali	35
4.2 Architettura generale del sistema	36
4.3 Algoritmi di sketching	37
4.3.1 Criteri di scelta	37
4.3.2 Interfaccia comune	38
4.3.3 Scelte progettuali comuni	38
4.3.4 Implementazione dei singoli algoritmi	39
4.4 Funzioni di hash	40
4.4.1 Ruolo dell'hash negli sketch	40
4.4.2 Interfaccia comune delle hash	41

4.4.3	Funzioni scelte e motivazione	41
4.4.4	Seed e riproducibilità	41
4.5	Framework sperimentale	42
4.5.1	Requisiti del framework	42
4.5.2	Struttura del framework	43
4.5.3	Flussi di esecuzione	43
4.5.4	Descrittori di esecuzione e formato dei risultati	44
4.5.5	Motivazioni architetturali	45
4.6	Dataset e riproducibilità	45
4.6.1	Formato binario e caricamento per partizione	46
4.6.2	Bitset di verità e cardinalità di prefisso	46
4.6.3	Protocollo <code>prefix_constant_rho</code>	47
4.6.4	Riproducibilità del dataset	47
4.7	Validazione	47
4.7.1	Test sugli algoritmi	48
4.7.2	Test del framework	48
4.7.3	Test del generatore e della catena di esecuzione	48
5	RISULTATI SPERIMENTALI	49
5.1	Validazione del contesto sperimentale	49
5.1.1	Verifica empirica del dataset a prefisso costante	50
5.1.2	Scelta del seed	50
5.2	Esperimenti sugli algoritmi	50
5.2.1	Configurazione utilizzata	53
5.2.2	Analisi della robustezza degli algoritmi	53
	Stima in funzione della cardinalità reale del prefisso	54
	Stima in funzione del rapporto tra elementi processati e distinti	54
	Risultati sulla robustezza e sul bias	54
5.2.3	Analisi sulla variazione dei parametro degli algoritmi	57
	Risultati sulla precisione per ogni algoritmo	59
5.3	Esperimenti sul merge distribuito	59
5.3.1	Caso omogeneo	60
5.3.2	Caso eterogeneo	60
	Hash mismatch	61
	Precision mismatch	62
5.4	Risultati ottenuti	65
6	CONCLUSIONI	67
6.1	Sviluppi futuri	68
	BIBLIOGRAFIA	71

Notazioni e concetti preliminari

In questa sezione si raccolgono le notazioni utilizzate nel resto della tesi.

- $\mathcal{U}$: universo degli elementi.
- $S = \langle x_1, \dots, x_s \rangle$: flusso di dati.
- $S_1 \cdot S_2$: concatenazione di due flussi.
- n : numero totale di elementi osservati; nei capitoli sperimentali coincide con la lunghezza della partizione o del dataset considerato. Nella discussione teorica, quando si riportano risultati della letteratura, il ruolo di parametro di scala del dominio è esplicitato localmente.
- $|\mathcal{U}|$: cardinalità dell'universo.
- $f(a)$: frequenza di $a \in \mathcal{U}$.
- (a_t, Δ_t) : aggiornamento al tempo t , con chiave a_t e incremento Δ_t .
- $A_t[j]$: frequenza dell'elemento j dopo i primi t aggiornamenti.
- $A_0[j] = 0$: condizione iniziale del modello a soli inserimenti.
- $f \in \mathbb{N}^{|\mathcal{U}|}$: vettore delle frequenze (componenti $f(a)$).
- $\|f\|_1 = \sum_{a \in \mathcal{U}} f(a)$: lunghezza totale del flusso.
- F_k : momenti di frequenza.
- F_0 : numero di distinti nel flusso.
- $\hat{F}_0$: stima di F_0 prodotta da un algoritmo.
- θ : quantità d'interesse da stimare.
- $\hat{\theta}$: stima della quantità θ .
- $\hat{\theta}(\mathcal{K}(S))$: stima della quantità θ quando si vuole esplicitare la dipendenza dalla struttura riassuntiva costruita sul flusso S .
- F_0^* : valore obiettivo della cardinalità distinta usato per interpolare i grafici della modalità di flusso.
- $\bar{F}_0$: media dei valori veri su R repliche.
- $\bar{\bar{F}}_0$: media delle stime su R repliche.
- μ : media della quantità osservata, usata localmente nei grafici con bande di dispersione.
- (ε, δ) : parametri di accuratezza; ε è l'errore relativo ammesso e $1 - \delta$ è la probabilità di successo.

- m : memoria della struttura riassuntiva (tipicamente espressa in bit); nelle sezioni su LogLog/HLL/HLL++ il numero di registri è indicato anch'esso con $m = 2^p$, quindi lo spazio complessivo dipende anche dalla larghezza di ciascun registro.
- M : stato interno dell'algoritmo per flussi di dati.
- $\mathcal{K}(\cdot)$: procedura che costruisce una struttura riassuntiva da un flusso.
- V : dominio di uscita di una funzione di hash.
- $h : \mathcal{U} \rightarrow V$: funzione di hash.
- $\mathcal{H}$: famiglia di funzioni di hash.
- L : lunghezza in bit del valore hash (nel Capitolo 3 si assume $L = w$).
- w : numero di bit del valore hash (se $V = \{0, 1\}^w$).
- p : numero di partizioni nei dataset sperimentali; nelle sezioni sugli sketch a registri lo stesso simbolo è usato localmente anche come parametro di precisione, con $m = 2^p$.
- p' : precisione usata nella rappresentazione sparsa di HLL++.
- k_{sp} : numero di voci non nulle nella rappresentazione sparsa di HLL++.
- d : numero di elementi distinti nel dataset o nella partizione considerata; nel Count-Min Sketch lo stesso simbolo è usato localmente per indicare il numero di righe.
- $\rho = \frac{n}{d}$: rapporto tra elementi totali e distinti nel dataset a prefisso costante.
- $\rho_t = t/F_0(t)$: rapporto tra elementi processati e distinti al tempo t .
- $\rho(\cdot)$: posizione del primo bit a 1; nel Capitolo 3 il significato preciso dipende dalla struttura considerata ed è ridefinito localmente dove necessario.
- $j(x)$: indice del registro selezionato dai $\log_2 m$ bit più significativi di $h(x)$.
- $w(x)$: suffisso di $h(x)$ usato per calcolare $\rho(w(x))$ nel registro $j(x)$.
- α_m : costante di normalizzazione dipendente da m .
- ϕ : costante di calibrazione di PCSA ($\phi \approx 0.77351$).
- $\beta(E, p)$: correzione empirica del bias usata in HyperLogLog++ per la precisione p .
- V_0 : numero di registri a zero (in HLL/HLL++).
- w_{cm} : numero di colonne nel Count-Min Sketch.
- m_{bf} : numero di bit del Bloom filter.
- k_{bf} : numero di funzioni hash del Bloom filter.
- n_{ins} : numero di elementi inseriti in un Bloom filter.
- p_{fp} : probabilità di falso positivo di un Bloom filter.

- $\mathcal{S}$: spazio degli stati di una struttura riassuntiva.
- $\oplus$: operatore di merge tra stati di strutture riassuntive.
- R : numero di repliche sperimentali.
- σ : deviazione standard campionaria delle stime.
- $\widehat{\text{Var}}(\hat{F}_0)$: varianza campionaria delle stime su R repliche.
- $\hat{\sigma}$: deviazione standard campionaria ($\hat{\sigma} = \sqrt{\widehat{\text{Var}}(\hat{F}_0)}$).
- RE: errore relativo.
- RSE: errore standard relativo.
- RSE_{obs} : errore standard relativo osservato, definito come $\hat{\sigma} / \bar{F}_0$.
- Δ_{abs} : differenza assoluta tra stima da merge e riferimento seriale.
- Δ_{rel} : differenza relativa tra stima da merge e riferimento seriale.
- D : rapporto di degrado tra errore del merge ed errore del riferimento seriale, valutato punto per punto e poi mediato sui casi osservati.
- $\text{Bias}(\hat{\theta})$: bias di uno stimatore.
- $\text{Var}(\hat{\theta})$: varianza di uno stimatore.
- SE: errore standard.
- $\widehat{\text{Bias}}(\hat{F}_0)$: stima empirica del bias su R repliche.
- AB: valore assoluto del bias empirico.
- RB: bias relativo.
- MRE: errore relativo medio.
- MAE: errore assoluto medio.
- MSE: errore quadratico medio.
- RMSE: radice dell'errore quadratico medio.

1

Introduzione

Il calcolo del numero di **elementi distinti** in un **flusso di dati** è un problema classico nell'analisi di grandi moli di dati e nei sistemi che ricevono aggiornamenti in modo continuo. Applicazioni come monitoraggio di rete, telemetria, osservabilità, analisi di log, ottimizzazione di interrogazioni e sistemi distribuiti richiedono spesso di *stimare quanti identificatori diversi siano comparsi in un flusso senza poter conservare l'intero input in memoria* [1, 2, 3]. Quando il *volume dei dati cresce*, il conteggio esatto *diventa costoso* sia in memoria sia in tempo, mentre il valore cercato deve spesso essere aggiornato con latenza molto bassa.

Da questa esigenza nasce l'uso di **strutture riassuntive probabilistiche**, indicate nel seguito come **sketch**, che comprimono il flusso in uno stato compatto e consentono di ottenere una stima approssimata del numero di distinti. La letteratura sul problema si è sviluppata lungo una linea piuttosto chiara, che parte dal conteggio probabilistico di Flajolet e Martin e arriva a *HyperLogLog++*, passando per *PCSA*, *LogLog* e *HyperLogLog* [4, 5, 6, 7]. Questa evoluzione migliora progressivamente il compromesso tra occupazione di memoria, accuratezza e costo di aggiornamento, ma lascia aperto un aspetto che nei sistemi distribuiti è centrale quanto la stima stessa: **l'unione di più sketch**.

Quando un flusso è *suddiviso tra più nodi* o tra più partizioni temporali, la stima globale non può essere ottenuta ricostruendo ogni volta l'insieme esatto degli elementi distinti. Diventa quindi *necessario combinare sketch costruiti localmente*. Nel caso ideale, cioè quando i due lati condividono la stessa struttura, la stessa semantica di hash e gli stessi parametri, l'operazione di **merge** è una proprietà naturale della struttura riassuntiva. Nei casi reali, però, possono comparire *eterogeneità* dovute a funzioni di hash diverse, seed diversi oppure precisioni diverse. In queste situazioni il merge non è più un'operazione banale: occorre capire quando sia semanticamente corretto, quando sia recuperabile tramite una normalizzazione esplicita e quando invece produca risultati privi di significato.

Il lavoro sviluppato si concentra proprio su questo punto. Da un lato *vengono ricostruite e implementate* in modo uniforme alcune delle principali famiglie di algoritmi per il conteggio approssimato degli elementi distinti. Dall'altro viene *realizzata un'infrastruttura sperimentale* che separa **algoritmi**, **funzioni di hash**, dataset e raccolta delle metriche, in modo da poter *confrontare i metodi* in un contesto simile ad un flusso e *studiare l'operazione di merge*.

in scenari distribuiti controllati. La parte sperimentale è organizzata in modo da osservare sia il comportamento della stima lungo il flusso sia quello dell'unione di sketch in caso omogeneo e nei principali casi eterogenei, con particolare attenzione a *HyperLogLog++* come riferimento pratico per lo studio dell'operazione di unione.

L'obiettivo non è introdurre un nuovo stimatore teorico, ma *comprendere in modo sistematico* il comportamento reale degli algoritmi implementati e delle condizioni di merge. Per fare questo vengono considerati sia il bias sia la robustezza della stima, viene costruito un dataset sintetico con proprietà controllate, e vengono osservati separatamente il ruolo della funzione di hash, del parametro di precisione e della compatibilità semantica tra sketch. In questo senso il contributo del lavoro è duplice: da un lato conferma in modo sperimentale proprietà attese della letteratura, dall'altro mette a disposizione un framework software riutilizzabile per lo studio di nuovi scenari di merge.

I.1 STRUTTURA DELLA TESI

Il Capitolo 2 introduce il modello dei flussi di dati, le funzioni di hash, le strutture riassuntive e le metriche statistiche usate nel seguito. Il Capitolo 3 ricostruisce lo sviluppo storico degli algoritmi per il conteggio approssimato degli elementi distinti e richiama anche altre strutture riassuntive impiegate per problemi diversi. Il Capitolo 4 descrive l'architettura del sistema sperimentale, le scelte progettuali, gli algoritmi implementati, le funzioni di hash e il protocollo di generazione dei dati. Il Capitolo 5 presenta i risultati sperimentali, prima in modalità streaming e poi negli esperimenti sul merge distribuito in caso omogeneo ed eterogeneo. Il Capitolo 6 raccoglie infine le considerazioni conclusive, i limiti del lavoro e le principali direzioni di sviluppo futuro.

2

Fondamenti teorici

A seguire vengono introdotte le nozioni teoriche utilizzate. Si parte dal **modello di streaming di dati**, si presentano **gli algoritmi per flussi di dati**, il ruolo delle **funzioni di hash** e delle strutture probabilistiche, infine si richiamano **i concetti statistici** necessari per interpretare le metriche sperimentali.

2.1 MODELLO DI STREAMING

Il **modello di streaming** descrive situazioni in cui gli elementi *arrivano in modo continuo e devono essere elaborati nell'ordine in cui vengono osservati* [1]. A differenza dei contesti tradizionali, **non si assume di poter memorizzare interamente l'input** prima di interrogarlo, di conseguenza *lo spazio disponibile è limitato e il numero totale di elementi può essere molto grande* o non noto a priori.

In questa sezione vengono introdotti, nell'ordine, gli oggetti di base del modello, il meccanismo di aggiornamento e la quantità principale studiata nella tesi.

Def. 2.1 (Flusso di dati). Sia $\mathcal{U}$ un universo di chiavi. Un *flusso di dati* è una sequenza ordinata

$$S = \langle x_1, x_2, \dots, x_s \rangle$$

dove ogni $x_i \in \mathcal{U}$ e s può essere molto grande o non noto a priori

Per descrivere il contenuto del flusso è utile introdurre le frequenze delle chiavi osservate.

Def. 2.2 (Frequenze). Data una sequenza S , la *frequenza* di un elemento $a \in \mathcal{U}$ è

$$f(a) = |\{i \mid x_i = a\}|$$

La **collezione delle frequenze** può essere vista come un vettore

$$f \in \mathbb{N}^{|\mathcal{U}|}$$

Nel modello generale per flussi di dati esistono diverse modalità di aggiornamento [2]. In questo contesto si adotta il caso **a soli inserimenti** (*insertion-only*), che è anche quello più naturale per il conteggio degli elementi distinti.

Def. 2.3 (Modello a soli inserimenti). Il flusso è una sequenza di aggiornamenti del tipo (a_t, Δ_t) , con $a_t \in \mathcal{U}$ e $\Delta_t \geq 0$. Indicando con $A_t[j]$ la frequenza dell'elemento j dopo i primi t aggiornamenti, vale

$$A_t[j] = \begin{cases} A_{t-1}[j] + \Delta_t & \text{se } a_t = j \\ A_{t-1}[j] & \text{altrimenti} \end{cases}$$

con inizializzazione $A_0[j] = 0$ per ogni $j \in \mathcal{U}$

Se il flusso è espresso direttamente come sequenza di valori, ogni elemento x_i può essere interpretato come l'aggiornamento $(x_i, 1)$.

Esistono modelli più generali, come il modello *turnstile*, in cui sono ammessi **aggiornamenti negativi** e il modello a finestra scorrevole (*sliding window*), in cui si considerano soltanto gli **ultimi** W aggiornamenti. Questi casi non rientrano nel nostro ambito, ma sono richiamati per collocare il problema nel contesto più ampio dello studio dei flussi di dati [2].

L'obiettivo principale considerato nel seguito è la stima del numero di elementi distinti presenti nel flusso.

Def. 2.4 (Numero di distinti). Il *numero di elementi distinti* del flusso S è

$$F_0 = |\{a \in \mathcal{U} \mid f(a) > 0\}|$$

Il numero di distinti è il caso particolare $k = 0$ di una famiglia di quantità più generale, i *momenti di frequenza* (*frequency moments*).

Def. 2.5 (Momenti di frequenza). Per ogni $k \geq 0$, il momento di frequenza F_k è definito come

$$F_k = \sum_{a \in \mathcal{U}} f(a)^k$$

In particolare, F_0 **coincide** con il numero di elementi distinti.

Nel problema affrontato in questa tesi, l'interesse è concentrato sul numero di distinti F_0 ma il richiamo ai momenti di frequenza è utile per mostrare che il conteggio degli elementi distinti è **parte di una classe più ampia** di problemi studiati nel modello dei flussi di dati.

2.2 ALGORITMI DI STREAMING

Il modello appena descritto rende *inapplicabili* gli approcci classici basati su memorizzazione completa e analisi a posteriori. Per questo motivo si introducono algoritmi che *elaborano il flusso in un solo passaggio* e mantengono *uno stato compatto* da aggiornare dopo ogni nuovo elemento.

Def. 2.6 (Algoritmo per flussi di dati). Un algoritmo per flussi di dati elabora un flusso in un solo passaggio e mantiene uno stato interno M di dimensione limitata. Per ogni elemento x_i si applica una funzione di aggiornamento

$$M \leftarrow \text{Update}(M, x_i)$$

e in qualunque momento è possibile ottenere una risposta o una stima tramite

$$\hat{\theta} \leftarrow \text{Query}(M)$$

dove θ è la quantità d'interesse associata al flusso osservato [2]

Questa astrazione separa il costo di aggiornamento per elemento dalla qualità della stima ottenuta interrogando lo stato. Nel seguito la quantità d'interesse principale sarà F_0 , ma la definizione resta generale.

Le prestazioni di un algoritmo per flussi di dati vengono tipicamente misurate in termini di numero di passaggi sul flusso, memoria impiegata, tempo per aggiornamento e accuratezza della risposta [8]. Nel caso di *algoritmi approssimati*, l'accuratezza è espressa tramite **parametri di errore e di confidenza**.

Def. 2.7 ((ε, δ) -approssimazione). Sia $\theta > 0$ una quantità d'interesse. Un algoritmo è detto (ε, δ) -*approssimante* per θ se produce una stima $\hat{\theta}$ tale che

$$\Pr(|\hat{\theta} - \theta| \leq \varepsilon\theta) \geq 1 - \delta$$

dove la probabilità è valutata rispetto alla randomizzazione interna dell'algoritmo [9]

Nel caso specifico del numero di elementi distinti, la definizione si applica alla scelta $\theta = F_0$ adottata nella tesi. Se il numero di elementi distinti è nullo, si richiede in modo naturale che la stima sia anch'essa nulla con alta probabilità.

L'altro vincolo fondamentale riguarda la memoria.

Def. 2.8 (Spazio sublineare). Un algoritmo usa *spazio sublineare* se la memoria m cresce asintoticamente meno di $n (= |\mathcal{U}|)$:

$$m = o(n)$$

Per il problema del conteggio degli elementi distinti esistono algoritmi che producono una $(1 \pm \varepsilon)$ -approssimazione usando spazio $\tilde{O}(\varepsilon^{-2} \log(1/\delta) + \log n)$ bit [10, 11]

I corrispondenti **limiti inferiori** mostrano che questa dipendenza è **ottimale** a livello di ordine di grandezza [11]

Ne segue un compromesso molto chiaro: ridurre ε di un fattore due richiede circa quattro volte la memoria, mentre ridurre δ richiede soltanto un fattore logaritmico, ottenibile anche tramite *ripetizioni indipendenti e combinazione delle risposte* [12].

La Figura 2.1 riassume la sequenza logica del modello: gli elementi del flusso aggiornano uno stato compatto, e da questo stato viene poi estratta la stima della quantità d'interesse.



Figura 2.1: Schema concettuale di un algoritmo per flussi di dati

È utile distinguere gli algoritmi per flussi di dati dagli algoritmi in linea (*online*). Entrambi operano senza disporre in anticipo dell'intero input, ma il modello dei flussi di dati è centrato soprattutto su **memoria sublineare** e **approssimazione**, e può anche ammettere l'elaborazione di piccoli blocchi di dati pur mantenendo uno stato molto compatto [2, 8]

In questo specifico caso, tutti gli algoritmi considerati elaborano ogni elemento appena arriva e sono quindi anche algoritmi online. Per ottenere questo comportamento servono, in pratica, meccanismi di randomizzazione efficienti, tipicamente realizzati tramite funzioni di hash.

2.3 FUNZIONI DI HASH

Per rispettare i vincoli di tempo e spazio appena descritti, gli algoritmi per flussi di dati impiegano spesso funzioni di hash che trasformano le chiavi del dominio in valori pseudo-casuali appartenenti a un dominio finito [13, 14]. Nel caso degli sketch per il conteggio degli elementi distinti, *la qualità della stima dipende in modo diretto dalla qualità della funzione di hash utilizzata*.

Def. 2.9 (Funzione di hash). Una funzione di hash è una funzione deterministica

$$h : \mathcal{U} \rightarrow V$$

dove $\mathcal{U}$ è l'universo delle chiavi e V è un dominio finito di uscita

L'uso di un dominio finito rende inevitabili le collisioni, cioè i casi in cui chiavi distinte producono lo stesso valore di hash. Per questo motivo una buona funzione di hash deve essere veloce da calcolare e distribuire i valori in modo quanto più possibile uniforme sul dominio di uscita.

La Figura 2.2 mostra in modo intuitivo come una funzione di hash possa essere vista come un meccanismo di partizionamento delle chiavi in un numero finito di celle o registri.

Le **proprietà** rilevanti per il seguito sono soprattutto *l'uniformità*, *la bassa probabilità di collisione*, *l'indipendenza* tra immagini di chiavi diverse e *l'efficienza di calcolo*.

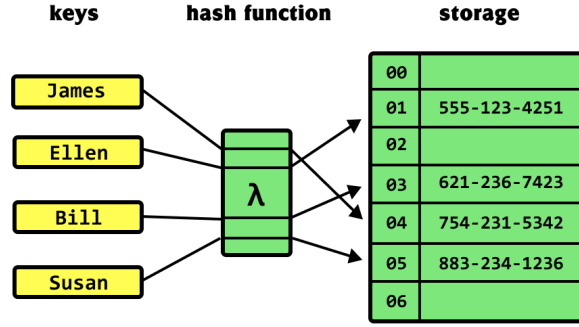


Figura 2.2: Schema intuitivo del partizionamento tramite hash

Def. 2.10 (Modello di hashing uniforme). Nel modello ideale, h è scelta **uniformemente** a caso dall'insieme di tutte le funzioni da $\mathcal{U}$ a V . In questo caso, per ogni chiave $x \in \mathcal{U}$ il valore $h(x)$ è uniforme in V e le immagini di chiavi distinte sono indipendenti [13, 14]

Def. 2.11 (Famiglia universale). Una famiglia $\mathcal{H}$ di funzioni $\mathcal{U} \rightarrow V$ è *universale* se per ogni coppia di chiavi distinte $x \neq y$ vale

$$\Pr_{h \leftarrow \mathcal{H}} [h(x) = h(y)] \leq \frac{1}{|V|}$$

[15]

Def. 2.12 (k -wise indipendenza). Una famiglia $\mathcal{H}$ è *k -wise indipendente* se, per ogni scelta di k chiavi distinte $x_1, \dots, x_k$, il vettore

$$(h(x_1), \dots, h(x_k))$$

è *distribuito uniformemente* in V^k quando h è scelta a caso da $\mathcal{H}$ [13]

Nelle analisi teoriche si assume spesso il modello ideale di hashing uniforme, oppure famiglie con gradi limitati di indipendenza. In pratica si usano funzioni più semplici, ma una scarsa uniformità può introdurre collisioni sistematiche e correlazioni tra registri, con conseguente aumento del bias e della varianza degli stimatori [13, 14].

Le funzioni di hash forniscono quindi il meccanismo con cui gli algoritmi trasformano il flusso in una rappresentazione compatta. La struttura che conserva questa rappresentazione è la struttura riassuntiva probabilistica, comunemente chiamata *sketch*.

2.4 SKETCH: STRUTTURE DATI RIASSUNTIVE

Una struttura riassuntiva probabilistica, o *sketch*, è una **struttura dati che mantiene una sintesi compatta del flusso senza conservarne esplicitamente tutti gli elementi** [3]. Lo stato della struttura riassuntiva viene aggiornato man mano che arrivano nuovi dati ed è poi interrogato per ottenere una stima della quantità d'interesse.

Nel seguito è importante non solo che una struttura riassuntiva possa essere aggiornata in modo efficiente, ma anche che possa essere **combinata con altre strutture riassuntive** costruite su porzioni diverse del flusso. Questa proprietà è **centrale nei contesti distribuiti**, in cui più nodi elaborano partizioni diverse dei dati e devono poi produrre una *stima globale*.

Def. 2.13 (Mergeable Sketch). Sia $\theta(S)$ la quantità d'interesse associata al flusso S , sia $\mathcal{K}(\cdot)$ la procedura che costruisce una struttura riassuntiva, sia $\hat{\theta}(\mathcal{K}(S))$ la stima prodotta dalla struttura riassuntiva costruita su S , e sia $\oplus$ un operatore definito sullo stato. Una struttura riassuntiva è *mergeabile* se, per due flussi S_1 e S_2 , vale

$$\hat{\theta}(\mathcal{K}(S_1) \oplus \mathcal{K}(S_2)) \approx \hat{\theta}(\mathcal{K}(S_1 \cdot S_2))$$

cioè se il merge di due strutture riassuntive consente di *ottenere la stessa informazione* che si otterrebbe elaborando **serialmente** la *concatenazione* dei due flussi, con le **stesse garanzie di errore** oppure con una degradazione esplicitamente caratterizzata [16].

Come nei casi precedenti la quantità d'interesse coincide con il numero di distinti F_0 considerato nel seguito. In questo contesto, **la concatenazione dei flussi** preserva **esattamente** l'informazione rilevante per il conteggio degli elementi distinti.

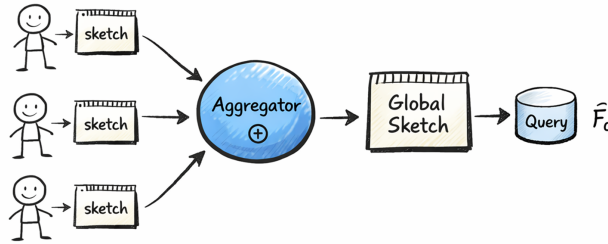


Figura 2.3: Merge di sketch in un contesto distribuito

La Figura 2.3 illustra lo scenario tipico in cui più nodi producono sketch locali e li aggregano gerarchicamente.

Def. 2.14 (Operatore chiuso). Sia $\mathcal{S}$ lo spazio degli stati di una struttura riassuntiva. Un operatore $\oplus$ è *chiuso* se:

$$\oplus : \mathcal{S} \times \mathcal{S} \rightarrow \mathcal{S}$$

la combinazione di due stati produce ancora uno stato valido dello stesso tipo.

Per aggregazioni robuste è inoltre desiderabile che l'operatore di merge sia *commutativo* e *associativo*. In molti casi è utile anche *l'idempotenza*, perché permette di tollerare duplicazioni accidentali dei contributi. Nel contesto dei sistemi distribuiti questo è essenziale, poiché, i nodi di uno sistema sono spesso “*stateless*”, ergo non hanno memoria di quello che è già stato fatto. Questa proprietà rende molto resilienti i sistemi distribuiti con questa proprietà.

Per gli sketch a registri considerati in questa tesi, come LogLog, HyperLogLog e HyperLogLog++, il **merge** che viene utilizzato, implementato seguendo la letteratura, coincide con **la funzione di massimo** applicata componente-per-componente. Questa operazione è **semanticamente corretta** solo quando gli sketch **condividono parametri strutturali** compatibili e la **stessa semantica di hash**.

Una volta definito l'operatore, conviene distinguere due assi concettualmente separati: la compatibilità semantica della coppia di sketch e la strategia operativa scelta dal sistema una volta nota tale compatibilità.

Def. 2.15 (Compatibilità semantica del merge). Nel seguito di questa tesi, una coppia di sketch è classificata in una tra queste tre categorie.

- **valid**: il merge è semanticamente corretto così com'è. Nel caso degli sketch della famiglia HLL, questo significa che l'operazione di massimo componente-per-componente rappresenta davvero la stessa unione che si otterrebbe elaborando serialmente i dati con la stessa specifica di sketch. Un caso tipico è una coppia di sketch HLL++ con la stessa precisione e la stessa semantica di hash.
- **recoverable**: la coppia non è direttamente mergeabile, ma può diventarlo dopo una normalizzazione esplicita e ben definita. Nel contesto della tesi, il caso tipico è HLL++ con precisioni diverse ma stessa semantica di hash, da riportare prima a una precisione comune.
- **invalid**: nel perimetro della tesi non esiste un merge semanticamente corretto della coppia. È il caso, ad esempio, di sketch costruiti con funzioni di hash diverse oppure di famiglie differenti.

Def. 2.16 (Strategia di merge). La strategia di merge è la politica operativa scelta dal sistema una volta nota la compatibilità semantica della coppia.

- **direct**: applica direttamente il merge nativo dello sketch
- **reject**: non tenta il merge e registra il caso come non supportato o incompatibile
- **reduce_then_merge**: applica prima una normalizzazione e poi il merge nativo
- **unsafe_naive_merge**: forza il merge come se la coppia fosse valida, anche quando non lo è, con finalità esclusivamente sperimentali

Questa distinzione sarà ripresa nei capitoli sperimentali per analizzare in modo sistematico il comportamento del merge nei casi omogenei ed eterogenei.

2.5 FAMIGLIA DI ALGORITMI PER IL CONTEGGIO DEGLI ELEMENTI DISTINTI

Il conteggio esatto di F_0 in un flusso di dati richiede, nel caso generale, memoria proporzionale al numero di chiavi distinte osservate. Gli algoritmi basati su strutture riassuntive rinunciano all'esattezza e producono una stima $\hat{F}_0$ con garanzie probabilistiche usando spazio sublineare e aggiornamenti a costo costante [9].

Una linea storica fondamentale parte dal *Probabilistic Counting* di Flajolet e Martin [4], che utilizza funzioni di hash per trasformare le chiavi in sequenze pseudo-casuali e ricava la cardinalità *osservandone i pattern nei bit*.

LogLog [5] e HyperLogLog [6] introducono poi una *struttura a registri*, ottenuti partizionando il dominio tramite hash, in cui ogni aggiornamento memorizza *informazioni sulla posizione del primo bit a uno* nel **suffisso dell'hash**. L'aggregazione dell'informazione su più registri riduce la varianza della stima.

HyperLogLog++ [7] mantiene la stessa idea di base, ma introduce **correzioni pratiche** per ridurre il bias in regimi diversi di cardinalità, in particolare nel caso di piccola cardinalità, e una doppia rappresentazione, sparsa e densa, per migliorare l'efficienza dell'uso della memoria.

Per HyperLogLog e HyperLogLog++, l'errore standard relativo (RSE) ha ordine $\Theta(1/\sqrt{m})$ rispetto al numero di registri m [6]

Questa relazione è importante anche dal punto di vista sperimentale, perché rende esplicito il compromesso tra spazio e accuratezza che verrà verificato nei capitoli successivi.

2.6 ALTRI OBIETTIVI DELLE STRUTTURE RIASSUNTIVE

Sebbene il focus della tesi sia il conteggio degli elementi distinti, lo stesso quadro concettuale si estende anche ad altri problemi. Esempi classici sono il Count-Min Sketch per la stima di frequenze e il Bloom filter per la verifica approssimata di appartenenza [17, 18, 19]

Questi casi verranno richiamati nel capitolo successivo per collocare la famiglia di algoritmi trattati nel panorama più ampio delle strutture riassuntive probabilistiche.

2.7 STIMATORI

Per interpretare correttamente le stime prodotte dalle strutture riassuntive è utile richiamare alcune nozioni elementari della teoria della stima. Nel seguito, **la stima prodotta** da una struttura riassuntiva viene **trattata** come **variabile aleatoria**, perché dipende sia dal flusso osservato sia dalla randomizzazione interna dell'algoritmo [20].

Quando serve esplicitare la dipendenza dalla struttura riassuntiva costruita sul flusso S scriveremo $\hat{\theta}(\mathcal{K}(S))$ in forma esplicita. In assenza di ambiguità, si userà la notazione più compatta $\hat{\theta}$ nel resto della sezione.

Def. 2.17 (Stimatore). Sia θ un parametro d'interesse e siano X i dati osservati. Uno *stimatore* è una funzione misurabile T tale che

$$\hat{\theta} = T(X)$$

dove $\hat{\theta}$ è una variabile aleatoria [20]

Def. 2.18 (Bias e correttezza). Il *bias* di uno stimatore è

$$\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta$$

Lo stimatore è *corretto* se il bias è nullo.

Un *bias positivo* indica una *tendenza sistematica a sovrastimare il valore vero*, mentre un *bias negativo* indica una *sottostima*. Un *bias nullo*, tuttavia, **non garantisce** da solo che *la stima sia stabile*.

Def. 2.19 (Varianza ed errore standard). La *varianza* di uno stimatore è

$$\text{Var}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2]$$

L'*errore standard* è la sua radice quadrata

$$\text{SE}(\hat{\theta}) = \sqrt{\text{Var}(\hat{\theta})}$$

La varianza misura quanto le stime oscillano tra esecuzioni indipendenti. Valori piccoli indicano maggiore stabilità, mentre valori grandi indicano una dispersione elevata.

Def. 2.20 (Errore di stima e rischio). L'*errore di stima* è la variabile aleatoria

$$E = \hat{\theta} - \theta$$

Data una funzione di perdita $L(E)$, il *rischio* è

$$R(\hat{\theta}) = \mathbb{E}[L(E)]$$

Scelte comuni per la funzione di perdita sono il valore assoluto e il quadrato dell'errore.

Def. 2.21 (Errore assoluto medio ed errore quadratico medio). L'*errore assoluto medio* (MAE) è

$$\text{MAE} = \mathbb{E}[|\hat{\theta} - \theta|]$$

L'*errore quadratico medio* (MSE) è

$$\text{MSE} = \mathbb{E}[(\hat{\theta} - \theta)^2]$$

Il MAE è facilmente interpretabile come distanza media dalla verità, mentre l'MSE penalizza maggiormente gli errori più grandi.

Def. 2.22 (Errore relativo). L'*errore relativo* è definito come

$$\text{RE} = \frac{|\hat{\theta} - \theta|}{|\theta|}$$

quando $\theta \neq 0$

Questa normalizzazione rende confrontabili stime prodotte su problemi di scala diversa.

Def. 2.23 (Consistenza). Una successione di stimatori $\hat{\theta}_n$ è *consistente* se

$$\hat{\theta}_n \xrightarrow{P} \theta \quad \text{quando } n \rightarrow \infty$$

La consistenza esprime il fatto che, al crescere dell'informazione disponibile, la stima converge al valore vero in probabilità. A partire da queste definizioni teoriche, si introducono ora le metriche empiriche usate nei capitoli sperimentali.

2.8 METRICHE EMPIRICHE DI VALUTAZIONE

Le definizioni precedenti sono di natura teorica. Nei capitoli sperimentali si useranno invece grandezze empiriche calcolate su repliche indipendenti. Nel seguito si torna al caso specifico del numero di distinti e quindi la quantità d'interesse coincide con F_0 nelle formule che seguono.

Per ogni replica $r = 1, \dots, R$ con stima $\hat{F}_0^{(r)}$ e valore vero $F_0^{(r)}$ si considerano l'errore di stima, il suo valore assoluto e l'errore relativo

$$\begin{aligned} e^{(r)} &= \hat{F}_0^{(r)} - F_0^{(r)} \\ |e^{(r)}| & \\ \text{RE}^{(r)} &= \frac{|e^{(r)}|}{F_0^{(r)}} \quad \text{se } F_0^{(r)} > 0 \end{aligned}$$

Quando le repliche sono eseguite su casi diversi ma confrontabili, è utile introdurre le medie dei valori veri e delle stime

$$\bar{F}_0 = \frac{1}{R} \sum_{r=1}^R F_0^{(r)} \quad \bar{\hat{F}}_0 = \frac{1}{R} \sum_{r=1}^R \hat{F}_0^{(r)}$$

La varianza campionaria delle stime è

$$\widehat{\text{Var}}(\hat{F}_0) = \frac{1}{R-1} \sum_{r=1}^R (\hat{F}_0^{(r)} - \bar{\hat{F}}_0)^2$$

e la corrispondente deviazione standard campionaria è

$$\hat{\sigma} = \sqrt{\widehat{\text{Var}}(\hat{F}_0)}$$

Il bias empirico, il suo valore assoluto e il bias relativo sono

$$\begin{aligned} \widehat{\text{Bias}}(\hat{F}_0) &= \bar{\hat{F}}_0 - \bar{F}_0 \\ \text{AB} &= |\widehat{\text{Bias}}(\hat{F}_0)| \\ \text{RB} &= \frac{\widehat{\text{Bias}}(\hat{F}_0)}{\bar{F}_0} \quad \text{se } \bar{F}_0 \neq 0 \end{aligned}$$

Tra le metriche aggregate usate nei risultati rientrano l'errore relativo medio (MRE), l'errore assoluto medio (MAE) e la radice dell'errore quadratico medio (RMSE)

$$\begin{aligned} \text{MRE} &= \frac{1}{R} \sum_{r=1}^R \frac{|\hat{F}_0^{(r)} - F_0^{(r)}|}{F_0^{(r)}} \\ \text{MAE} &= \frac{1}{R} \sum_{r=1}^R |\hat{F}_0^{(r)} - F_0^{(r)}| \end{aligned}$$

$$\text{RMSE} = \sqrt{\frac{1}{R} \sum_{r=1}^R (\hat{F}_0^{(r)} - F_0^{(r)})^2}$$

Per collegarsi alla letteratura sulle strutture riassuntive è utile considerare anche l'errore standard relativo osservato, indicato con RSE_{obs}

$$\text{RSE}_{\text{obs}} = \frac{\hat{\sigma}}{\bar{F}_0}$$

nel caso in cui $\bar{F}_0 > 0$. Se la cardinalità media è nulla, anche la deviazione standard campionaria è nulla e l'errore standard relativo osservato viene posto convenzionalmente uguale a zero.

Quando il valore vero è lo stesso in tutte le repliche, questa definizione si riduce al rapporto $\hat{\sigma}/F_0$ usato come riferimento comune. Quando invece $F_0^{(r)}$ varia tra le repliche, il termine RSE_{obs} va interpretato come normalizzazione rispetto alla scala media del problema. Nei capitoli sperimentali questa quantità sarà confrontata, quando disponibile, con formule teoriche del tipo $c/\sqrt{m}$ per verificare la coerenza tra predizioni asintotiche e risultati osservati.

3

Analisi dello stato dell'arte

Nel Capitolo 2 sono state introdotte le nozioni teoriche che servono per poter comprendere il problema del conteggio degli elementi distinti nei flussi di dati. Ora si vuole analizzare la parte algoritmica relativa al problema. L'obiettivo è ricostruire lo sviluppo che porta dal conteggio probabilistico originario di Flajolet e Martin fino a HyperLogLog++, chiarendo per ogni metodo come è fatto lo stato, come avviene l'aggiornamento, quale stima produce, quale costo richiede e in che modo si presta al merge di sketch. Si esaminano nell'ordine Probabilistic Counting, PCSA, LogLog, HyperLogLog e HyperLogLog++. Per ciascun algoritmo si descrivono stato, aggiornamento, stima, complessità e proprietà di merge. Infine si richiamano Bloom filter e Count-Min Sketch come esempi di sketch usati per obiettivi diversi dal solo conteggio degli elementi distinti.

3.1 OBIETTIVO DEL PROBLEMA E VINCOLI TEORICI

Il problema del conteggio degli elementi distinti consiste nello stimare

$$F_0 = |\{a \in \mathcal{U} : f(a) > 0\}|$$

ossia la cardinalità del supporto del vettore delle frequenze. Nel quadro dei *frequency moments*, F_0 rappresenta il primo problema di stima con memoria sublineare sui flussi di dati [10, 9]

In un modello *insertion-only*, un algoritmo deve processare ogni elemento in uno o pochi passaggi, con tempo di aggiornamento molto basso e stato ridotto. Il calcolo esatto richiede in generale memoria lineare nel numero di distinti osservati e quindi non può scalare quando il flusso cresce senza un limite prefissato. Per questo motivo la letteratura ricorre a sketch randomizzati con garanzie probabilistiche (ε, δ) , già formalizzate nel Capitolo 2

Dal lato teorico esistono algoritmi ottimali che raggiungono spazio dell'ordine $O((\varepsilon^{-2} + \log n) \text{polylog } n)$, mostrando che la dipendenza da ε^{-2} è strutturale e non una conseguenza implementativa [11]. Questo, però è

un risultato moderno, utile come termine di paragone: gli algoritmi studiati nel seguito nascono prima di questi risultati ottimali e vanno letti come tappe storiche che migliorano progressivamente il compromesso tra spazio e accuratezza.

Nel seguito gli algoritmi vengono confrontati secondo sette criteri:

- forma dello stato dello sketch;
- regola di aggiornamento;
- forma dello stimatore;
- errore tipico o comportamento dell'errore;
- costo in spazio;
- tempo di aggiornamento e di interrogazione;
- proprietà di merge di sketch in scenari distribuiti.

3.2 SVILUPPO STORICO

Lo sviluppo storico degli algoritmi per il conteggio degli elementi distinti segue un percorso piuttosto lineare. Si parte da un vettore di bit, chiamato bitmap, che tiene conto della frequenza di eventi poco frequenti, per poi passare a una mediazione stocastica su più bitmap. A seguire una famiglia di registri che usa una funzione di massimo sul rango dei registri, quindi a una stima armonica con correzioni di intervallo, e infine a una variante pratica che aggiunge correzione del bias e rappresentazione sparsa.

Il primo algoritmo che andiamo ad analizzare è Probabilistic Counting.

Def. 3.1 (Probabilistic Counting). Sia $h : \mathcal{U} \rightarrow \{0, 1\}^L$ una funzione di hash uniforme. Nella versione elementare del metodo si mantiene una sola bitmap $B = (b_1, \dots, b_L) \in \{0, 1\}^L$ inizializzata a zero.

Per ogni valore hash y , si considera la quantità $\rho(y)$, che rappresenta la posizione della bitmap da porre a uno. In altre parole, $\rho(y)$ misura quanti zeri consecutivi compaiono a partire dal bit meno significativo di y , più uno. Nella convenzione usata qui si pone

$$\rho(y) = \min\{1 + \nu_2(y), L\}$$

dove $\nu_2(y)$ è il numero di zeri consecutivi a partire dal bit meno significativo.

Per ogni elemento x del flusso si aggiorna quindi la bitmap ponendo

$$b_{\rho(h(x))} \leftarrow 1$$

Indicando con

$$R(B) = \min(\{r \geq 1 : b_r = 0\} \cup \{L + 1\})$$

la posizione del primo zero nella bitmap, con la convenzione che se la bitmap è interamente a uno si ponga $R(B) = L + 1$, la stima assume la forma

$$\hat{F}_0 \approx \frac{2^{R(B)}}{\phi}$$

dove ϕ è una costante di calibrazione empirica [4]

La struttura è estremamente compatta e mostra già il principio fondamentale del conteggio probabilistico, ma una sola bitmap produce una varianza elevata. Per questa ragione la versione elementare è importante soprattutto come punto di partenza concettuale e come riferimento storico.

Algoritmo 3.1 Probabilistic Counting

```

Choose a bitmap  $B[1], \dots, B[L]$  and initialize it to 0
Choose a uniform hash function  $h : \mathcal{U} \rightarrow \{0, 1\}^L$ 
for each element  $x$  in the stream
     $y \leftarrow h(x)$ 
     $r \leftarrow \rho(y)$ 
     $B[r] \leftarrow 1$ 
end for
 $R \leftarrow \text{FIRSTRIGHTMOSTZERO}(B)$ 
return  $\hat{F}_0 \leftarrow 2^R / \phi$ 

```

ESEMPIO Supponiamo di usare una bitmap di lunghezza $L = 5$. Consideriamo la sequenza di valori hash $00101_2, 00010_2, 11100_2, 01000_2$

Nel primo caso $\rho = 1$, nel secondo $\rho = 2$, nel terzo $\rho = 3$ e nel quarto $\rho = 4$. Dopo questi quattro aggiornamenti la bitmap vale

$$B = [11110]$$

quindi il primo zero si trova in posizione $R(B) = 5$ e la stima è circa $2^5 / \phi$. L'esempio è volutamente minimo, ma mostra bene la logica del metodo: gli eventi più rari osservati sull'hash vengono trasformati in una scala di cardinalità.

La versione a bitmap singola è molto soggetta alla varianza. Il vantaggio è la semplicità, ma l'errore resta comunque troppo elevato per un uso pratico effettivo. È proprio da questa debolezza che nasce il passo successivo, cioè *PCSA*.

Lo spazio utilizzato dall'algoritmo è $\Theta(L)$ bit, mentre la complessità in tempo è $\Theta(1)$ per elemento. Il tempo di query richiede $\Theta(L)$ nel caso diretto, oppure tempo costante se si mantengono informazioni ausiliari sulla posizione del primo zero.

3.2.1 PROBABILISTIC COUNTING WITH STOCHASTIC AVERAGING

Nello stesso lavoro del 1985, Flajolet e Martin introducono *Probabilistic Counting with Stochastic Averaging* (**PCSA**) [4]. L'idea è non affidare più tutta la stima a una sola bitmap, ma distribuire l'informazione su più bitmap virtuali ottenute tramite partizione del flusso.

Def. 3.2 (PCSA). Siano m il numero di bitmap e L la loro lunghezza. Ogni elemento x viene trasformato in un valore hash $y = h(x)$ e da esso si ricavano:

$$j(x) = y \bmod m$$

$$t(x) = \lfloor y/m \rfloor$$

Il valore $j(x)$ seleziona la bitmap da aggiornare, mentre $t(x)$ viene usato per calcolare il rango $\rho(t(x))$. Per ciascuna bitmap $B[j]$ si calcola il primo zero R_j e la stima finale usa la media dei valori osservati:

$$\bar{R} = \frac{1}{m} \sum_{j=0}^{m-1} R_j$$

$$\hat{F}_0 \approx \frac{m}{\phi} \cdot 2^{\bar{R}}$$

In questo modo la partizione è virtuale e dipende soltanto dalla funzione di hash. Non vengono create più sequenze fisiche separate, ma più sottostrutture che raccolgono informazione in modo calcolato.

Sotto l'ipotesi che il valore usato per aggiornare ciascuna bitmap sia uniforme, la variabile $\rho(z)$ conserva coda geometrica:

$$\Pr[\rho(z) \geq \tau] = 2^{-(\tau-1)}$$

Questa proprietà è il fondamento probabilistico del metodo. PCSA conserva la stessa idea del conteggio probabilistico originario, ma la applica su più bitmap virtuali, riducendo la varianza della stima attraverso la media dei contributi osservati.

Algoritmo 3.2 PCSA

Choose m bitmaps $B[0], \dots, B[m-1]$ of length L and initialize them to 0

Choose a uniform hash function $h : \mathcal{U} \rightarrow \{0, 1\}^L$

for each element x in the stream

$y \leftarrow h(x)$

$j \leftarrow y \bmod m$

$t \leftarrow \lfloor y/m \rfloor$

$r \leftarrow \rho(t)$

$B[j, r] \leftarrow 1$

end for

for $j = 0$ **to** $m - 1$

$R_j \leftarrow \text{FIRSTRIGHTMOSTZERO}(B[j])$

end for

$\bar{R} \leftarrow \frac{1}{m} \sum_{j=0}^{m-1} R_j$

return $\hat{F}_0 \leftarrow \frac{m}{\phi} \cdot 2^{\bar{R}}$

ESEMPIO Consideriamo $m = 4$ bitmap di lunghezza 4 e rappresentiamo ogni bitmap come $[b_1b_2b_3b_4]$, dove $b_r = 1$ indica che la posizione r è stata osservata almeno una volta.

Passo	x	$y = h(x)$	$j = y \bmod 4$	$t = \lfloor y/4 \rfloor$	$r = \rho(t)$	Aggiornamento	Stato bitmap ($B_0 B_1 B_2 B_3$)	$\hat{F}_0$
1	a	25	1	6 (110_2)	2	$B[1, 2] \leftarrow 1$	0000 0100 0000 0000	≈ 10.34
2	b	14	2	3 (11_2)	1	$B[2, 1] \leftarrow 1$	0000 0100 1000 0000	≈ 12.29
3	c	9	1	2 (10_2)	2	$B[1, 2] \leftarrow 1$	0000 0100 1000 0000	≈ 12.29
4	d	7	3	1 (1_2)	1	$B[3, 1] \leftarrow 1$	0000 0100 1000 1000	≈ 14.62

Tabella 3.1: Esempio operativo di PCSA

La tabella mostra che gli elementi vengono distribuiti tra sottostrutture diverse. La stima finale non dipende più da una sola bitmap, ma dalla media dei primi zeri osservati in più bitmap.

Per PCSA la costante ϕ viene stimata empiricamente. L'errore relativo standard atteso è circa $0.78/\sqrt{m}$.

Gli autori riportano anche ordini di grandezza pratici: con $m = 64$ si ottiene tipicamente un errore intorno al 10%, mentre $m = 256$ si scende intorno al 5%

Lo spazio per questa struttura dati, dato m ed L è $\Theta(mL)$ bit.

L'aggiornamento della struttura richiede un tempo $\Theta(1)$ per elemento. Mentre fare una query sulla struttura richiede $\Theta(mL)$ nel caso diretto, oppure $\Theta(m)$ se si mantiene informazione ausiliaria sul primo zero di ciascuna bitmap.

Il limite principale di PCSA non è più solo la varianza, ma anche il costo di mantenere molte bitmap. Da qui nasce il passaggio a strutture a registri, introdotto da LogLog.

3.2.2 LOGLOG

LogLog sostituisce la bitmap con un array di registri e applica la mediazione stocastica in modo più efficiente [5]. Il prefisso dell'hash seleziona il registro da aggiornare, mentre il suffisso determina il rango ρ da propagare come massimo. La stima finale usa una media geometrica normalizzata. Nel lavoro originale, inoltre, compare anche SuperLogLog, una variante che riduce ulteriormente la dispersione scartando i registri più estremi e che costituisce il passaggio storico immediatamente precedente a HyperLogLog [5].

Def. 3.3 (LogLog). Sia $m = 2^p$ il numero di registri. Per ogni elemento x si calcola un valore di hash $y = h(x)$, si usa il prefisso di p bit per selezionare il registro j , e il suffisso w per calcolare il rango

$$\rho(w) = \begin{cases} \min\{r \geq 1 : w_r = 1\}, & \text{se esiste un bit a 1} \\ |w| + 1, & \text{altrimenti} \end{cases}$$

dove i bit del suffisso sono letti da sinistra verso destra

L'update della struttura viene fatto, quindi come:

$$M[j] \leftarrow \max\{M[j], \rho(w)\}$$

Indicando con

$$A = \frac{1}{m} \sum_{j=0}^{m-1} M[j]$$

lo stimatore LogLog ha forma

$$\hat{F}_0 = \alpha_m \cdot m \cdot 2^A$$

dove α_m è una costante di calibrazione (asintoticamente circa 0.397).

La relazione con F_0 può essere letta anche in modo esplicito: il numero atteso di elementi che cade in ciascun registro è circa F_0/m , quindi i massimi dei registri tendono a concentrarsi attorno a $\log_2(F_0/m)$.

La media dei registri fornisce quindi una stima logaritmica della cardinalità complessiva. Il passaggio fondamentale rispetto a Probabilistic Counting è che, a parità di memoria, la dispersione delle stime si riduce sensibilmente grazie all'aggregazione su molti registri.

Algoritmo 3.3 LogLog

```

Choose the precision  $p$  and set  $m = 2^p$ 
Initialize the registers  $M[0], \dots, M[m-1] \leftarrow 0$ 
for each element  $x$  in the stream
     $y \leftarrow h(x)$ 
     $j \leftarrow \text{PREFIX}_p(y)$ 
     $w \leftarrow \text{SUFFIX}_{L-p}(y)$ 
     $M[j] \leftarrow \max\{M[j], \rho(w)\}$ 
end for
 $A \leftarrow \frac{1}{m} \sum_{j=0}^{m-1} M[j]$ 
return  $\hat{F}_0 \leftarrow \alpha_m \cdot m \cdot 2^A$ 

```

ESEMPIO Consideriamo un caso minimale con $p = 2$, quindi $m = 4$ registri e hash su 8 bit. I primi due bit selezionano il registro i restanti sei bit formano il suffisso.

Passo	x	$y = h(x)$	j	w	$\rho(w)$	Aggiornamento	Stato registri (M_0, M_1, M_2, M_3)	$\hat{F}_0$
1	a	10010100 ₂	2	010100 ₂	2	$M[2] \leftarrow 2$	(0, 0, 2, 0)	≈ 2.25
2	b	10111000 ₂	2	111000 ₂	1	$M[2] \leftarrow 2$	(0, 0, 2, 0)	≈ 2.25
3	c	01001100 ₂	1	001100 ₂	3	$M[1] \leftarrow 3$	(0, 3, 2, 0)	≈ 3.78
4	d	11001000 ₂	3	001000 ₂	3	$M[3] \leftarrow 3$	(0, 3, 2, 3)	≈ 6.352
5	e	10000100 ₂	2	000100 ₂	4	$M[2] \leftarrow 4$	(0, 3, 4, 3)	≈ 8.98

Tabella 3.2: Esempio operativo di LogLog

Ogni aggiornamento modifica un solo registro, quindi il costo di aggiornamento resta costante. La differenza rispetto a PCSA è che l'informazione non viene più distribuita su bitmap, ma su registri che memorizzano un massimo.

L'errore relativo standard tipico è dell'ordine $1.30/\sqrt{m}$ [5].

Se ogni registro codifica valori in $[0, L - p + 1]$, servono $\lceil \log_2(L - p + 2) \rceil$ bit per registro. Quindi lo spazio, dato m, L, p è $\Theta(m \log(L - p + 2))$ bit. L'aggiornamento per elemento è $\Theta(1)$ e l'interrogazione sulla struttura dati è $\Theta(m)$.

LogLog riduce molto la varianza rispetto a PCSA, ma resta sensibile alla costante di normalizzazione e alla qualità della funzione di hash. Per cardinalità basse rispetto a m , il bias può essere ancora marcato.

La sensibilità residua di LogLog e la necessità di uno stimatore più stabile portano nel 2007 a HyperLogLog, che conserva i registri ma cambia la regola di stima.

3.2.3 HYPERLOGLOG

HyperLogLog mantiene la stessa struttura a registri di LogLog ma sostituisce la funzione di stima, passando dalla media geometrica alla media armonica delle quantità $2^{-M[j]}$ [6].

In questo modo si ottiene una migliore analizzabilità e una costante di errore migliore.

Def. 3.4 (HyperLogLog). Definendo

$$Z = \sum_{j=0}^{m-1} 2^{-M[j]}$$

la stima grezza di HLL è

$$E = \alpha_m \frac{m^2}{Z}$$

Per i valori di m più piccoli si usano costanti pratiche dedicate, definite come in [6], mentre per $m \geq 128$ si usa tipicamente

$$\alpha_m \approx \frac{0.7213}{1 + 1.079/m}$$

Se V_0 è il numero di registri a zero, questo algoritmo distingue tre regimi di quantità degli elementi in streaming e introduce delle correzioni nella stima:

$$\hat{F}_0 = \begin{cases} m \log(m/V_0), & \text{se } E \leq \frac{5}{2}m \text{ e } V_0 > 0 \\ E, & \text{nel regime centrale} \\ -2^L \log(1 - \frac{E}{2^L}), & \text{nel regime vicino a } 2^L \end{cases}$$

La scelta della media armonica attenua il peso dei registri estremi e rende la stima più stabile di quella ottenuta con LogLog. Dal punto di vista statistico, la varianza scende mantenendo la stessa struttura di aggiornamento, con un vantaggio diretto nel rapporto tra la memoria e l'accuratezza.

ESEMPIO Supponiamo $m = 16$ e che, dopo l'aggiornamento dello sketch, restino $V_0 = 10$ registri nulli. In questo caso il conteggio lineare produce

$$16 \log(16/10) \approx 7.52$$

Algoritmo 3.4 HyperLogLog

Choose the precision p and set $m = 2^p$
Initialize the registers $M[0 \dots m-1] \leftarrow 0$
for each element x in the stream
 $y \leftarrow h(x)$
 $j \leftarrow \text{PREFIX}_p(y)$
 $w \leftarrow \text{SUFFIX}_{L-p}(y)$
 $M[j] \leftarrow \max\{M[j], \rho(w)\}$
end for
 $Z \leftarrow \sum_{j=0}^{m-1} 2^{-M[j]}$
 $E \leftarrow \alpha_m m^2 / Z$
if $E \leq \frac{5}{2}m$
 $V_0 \leftarrow$ number of zero registers
 if $V_0 \neq 0$
 $E^* \leftarrow m \log(m/V_0)$
 else
 $E^* \leftarrow E$
 end if
else if $E \leq \frac{1}{30} \cdot 2^L$
 $E^* \leftarrow E$
else
 $E^* \leftarrow -2^L \log(1 - E/2^L)$
end if
return $\hat{F}_0 \leftarrow E^*$

Finché molti registri restano nulli, questa correzione è più affidabile della stima grezza E . Quando invece V_0 diminuisce, il metodo passa al regime centrale basato sulla media armonica.

Il risultato classico è una deviazione standard relativa tipica $1.04/\sqrt{m}$ [6].

Con $m = 2^p$ registri, lo spazio, dato m, L, p è $\Theta(m \log(L - p + 2))$ bit.

In pratica i registri vengono spesso implementati con ampiezza fissa di 5 o 6 bit. Il tempo di aggiornamento è $\Theta(1)$ per elemento e di query sulla struttura è $\Theta(m)$.

Le formule precedenti assumono parametrizzazione coerente dello sketch, stessa definizione di rango e funzione di hash sufficientemente uniforme. In presenza di hash distorto, i registri non campionano più correttamente il flusso e la stima può mostrare del bias non trascurabile.

Prima di passare a HyperLogLog++, conviene isolare un punto. HyperLogLog non coincide con una sola formula, ma con una procedura a regimi. Nei regimi piccoli si usa il conteggio lineare quando molti registri restano nulli, nel regime centrale si usa lo stimatore armonico, e vicino al limite del dominio hash si applica una correzione di saturazione. HyperLogLog++ parte da questa struttura, ma aggiunge una correzione empirica del bias e una rappresentazione sparsa per gestire meglio le cardinalità piccole e intermedie.

3.2.4 HYPERLOGLOG++

HyperLogLog++ nasce come evoluzione pratica di HLL in scenari industriali su larga scala. I miglioramenti principali sono l'uso di una funzione di hash a 64 bit (rispetto ai 32 bit precedenti), la rappresentazione sparsa (lista di vettori) nelle cardinalità piccole, la correzione empirica del bias e la scelta adattiva tra conteggio lineare e stima HLL corretta [7].

Def. 3.5 (HyperLogLog++). La struttura mantiene la stessa regola di aggiornamento di HLL sui registri, ma introduce due passaggi aggiuntivi: rappresentazione sparsa nelle cardinalità piccole e correzione empirica del bias.

Indicando con E la stima HLL grezza, la stima corretta è

$$E_{\text{corr}} = \begin{cases} E - \beta(E, p), & \text{se } E \leq 5m \\ E, & \text{altrimenti} \end{cases}$$

dove $\beta(E, p)$ è il termine di bias tabulato per la precisione p .

La scelta finale tra conteggio lineare e stima corretta usa una soglia empirica $\text{THRESHOLD}(p)$ applicata alla stima prodotta nei regimi di cardinalità bassa

La formulazione del lavoro del 2013 usa inoltre due precisioni: p per la rappresentazione densa e p' per la codifica sparsa, con $p \leq p' \leq 64$.

Nel formato sparso lo sketch memorizza soltanto i registri non nulli sotto forma di coppie indice-valore, in questo modo risparmiando spazio e avendo costo costante nelle cardinalità piccole. Quando questa forma non è più conveniente in memoria, lo sketch passa alla rappresentazione densa, cioè tramite l'utilizzo di matrici.

ESEMPIO Con $p = 14$, quindi $m = 16384$ registri, e cardinalità ancora bassa, lo sketch può restare in formato sparso. Se per esempio nella struttura sono presenti poche voci non nulle, lo spazio usato è molto inferiore rispetto

Algoritmo 3.5 HyperLogLog++

Input: stream S , precisions p and p' with $p \leq p' \leq 64$
 $m \leftarrow 2^p, \quad m' \leftarrow 2^{p'}$
Initialize sparse representation, temporary set, and dense registers
for each element x in S
 $y \leftarrow h_{64}(x)$
 if the structure is in sparse format
 Encode the sparse value associated with y
 Insert the value into the temporary set
 if the temporary set is too large
 Merge the temporary set into the sparse list
 end if
 if the sparse representation is no longer convenient
 Convert the sketch to dense format
 end if
 else
 Extract (j, ρ) from y
 $M[j] \leftarrow \max\{M[j], \rho\}$
 end if
end for
if the structure is in sparse format
 $k_{\text{sp}} \leftarrow$ number of nonzero entries in the sparse representation
 return LINEARCOUNTING($m', m' - k_{\text{sp}}$)
else
 Compute the raw estimate E
 if $E \leq 5m$
 $E_{\text{corr}} \leftarrow E - \beta(E, p)$
 else
 $E_{\text{corr}} \leftarrow E$
 end if
 $V_0 \leftarrow$ number of zero registers
 if $V_0 > 0$
 $H \leftarrow$ LINEARCOUNTING(m, V_0)
 else
 $H \leftarrow E_{\text{corr}}$
 end if
 if $H \leq \text{THRESHOLD}(p)$
 return H
 else
 return E_{corr}
 end if
end if

alla rappresentazione densa. Quando il numero di voci non nulle supera la soglia di conversione, lo sketch passa alla forma densa; da quel momento la stima viene calcolata correggendo il bias per i valori bassi di E e confrontando il conteggio lineare con la soglia empirica.

HyperLogLog++ mantiene l'ordine asintotico tipico della famiglia, cioè $\Theta(1/\sqrt{m})$, ma riduce in modo sensibile il bias pratico soprattutto nei regimi piccoli e intermedi.

La differenza rispetto a HLL non è nell'ordine dominante, ma nella qualità empirica della stima e nella gestione dei regimi di cardinalità.

In modalità sparsa, lo spazio è $\Theta(k_{sp})$, dove k_{sp} è il numero di registri non nulli, fino alla soglia di conversione. In modalità densa, lo spazio è $\Theta(m)$ registri, spesso intorno a $6m$ bit nelle implementazioni pratiche.

L'aggiornamento richiede $\Theta(1)$ ammortizzato, con costo $\Theta(m)$ nel momento della conversione da sparso a denso. L'interrogazione è $\Theta(k_{sp})$ in formato sparso e $\Theta(m)$ in formato denso.

HLL++ riduce il bias pratico e gestisce meglio le cardinalità piccole, ma introduce maggiore complessità ingegneristica: soglie di cambio di regime, tabelle di correzione e doppia rappresentazione dello stato. Per questa ragione, nel seguito della tesi sarà importante distinguere sempre la specifica algoritmica dai dettagli implementativi.

La letteratura successiva non si è però fermata a HLL++. Ertl propone nuovi stimatori per sketch HyperLogLog e una trattazione più fine di unione, intersezione e riduzione di precisione [21]; Lang introduce CPC/FM85 come alternativa pratica alla stessa classe di problemi [22]; lavori più recenti esplorano ulteriori evoluzioni come SetSketch, HyperLogLogLog e UltraLogLog [23, 24, 25]. Questo mostra che, anche dopo HLL++, la ricerca continua a muoversi lungo due direttrici complementari: migliorare la qualità dello stimatore e ridurre ulteriormente il costo in spazio.

3.3 MERGE DI ALGORITMI DI SKETCH E SCENARI DISTRIBUITI

Una proprietà fondamentale degli sketch di cardinalità moderni è la mergeabilità, cioè la possibilità di costruire sketch locali su partizioni del flusso e combinarli in uno sketch globale senza rivedere i dati originali [26, 27, 16]

Per Probabilistic Counting e PCSA, dove lo stato è basato su bitmap, la combinazione naturale è una OR componente per componente. Per LogLog, HLL e HLL++, mantenendo la notazione del Capitolo 2, la regola di merge naturale è

$$\forall j \in m(\mathcal{K}(S_1) \oplus \mathcal{K}(S_2))[j] = \max\{\mathcal{K}(S_1)[j], \mathcal{K}(S_2)[j]\}$$

Ne segue che, a parità di parametrizzazione e semantica di hashing, il merge registro per registro coincide con lo stato ottenuto processando l'unione dei flussi, che è precisamente la proprietà sfruttata nella letteratura per usare questi sketch in scenari distribuiti e per stimare quantità legate a unioni e intersezioni [28, 21].

Questa operazione è commutativa, associativa e idempotente, quindi è adatta agli ambiti distribuiti in cui la topologia, spesso, non è nota a priori.

Nel caso omogeneo, cioè stessa parametrizzazione e stessa semantica di hashing, questa operazione coincide con lo stato che si otterrebbe processando in modo sequenziale l'unione dei due flussi. Nel lessico introdotto nel Capitolo 2, questo è un caso semanticamente valido.

Il merge di sketch, però, non si esaurisce nel caso omogeneo. Nella tesi il punto più importante è che HyperLogLog++ permette di studiare anche casi eterogenei. Se cambia la precisione ma non cambia la semantica dell'hash,

si ha un caso semanticamente recoverable, in cui la riduzione alla precisione più bassa rende confrontabili gli sketch [21]. Se invece cambia la semantica di hashing, il caso è invalid. Questo è il collegamento diretto con il Capitolo 4, in cui queste situazioni vengono rese operative nell'infrastruttura sperimentale.

3.4 CONFRONTO SINTETICO TRA GLI APPROCCI

Dopo aver discusso anche i regimi di correzione e la mergeabilità, si può riassumere la traiettoria storica con due tabelle di confronto.

Algoritmo	Stato dello sketch	Regola di stima	Punti di forza	Limiti pratici
Probabilistic Counting	Una bitmap singola	Primo zero e costante di calibrazione	Metodo originario basato su bitmap	Varianza elevata, stima poco stabile
PCSA	Più bitmap partizionate tramite hash	Mediazione stocastica su bitmap	Riduce la varianza rispetto a una sola bitmap	Stato più pesante e meno compatto delle famiglie a registri
LogLog	Array di registri	Media geometrica normalizzata	Riduce la varianza usando registri	Sensibilità alla costante di normalizzazione e al regime di cardinalità bassa
HyperLogLog	Array di registri	Media armonica con correzioni di intervallo	Stima più stabile e merge naturale a registri	Richiede gestione distinta dei diversi regimi di cardinalità
HyperLogLog++	Registri con rappresentazione sparsa e densa	HLL corretto dal bias con soglia empirica tra regimi	Aggiunge correzione del bias e rappresentazione sparsa	Maggiore complessità ingegneristica

Tabella 3.3: Confronto qualitativo tra gli algoritmi principali per il conteggio degli elementi distinti

Algoritmo	Spazio	Aggiornamento	Interrogazione	Errore tipico
Probabilistic Counting	$\Theta(L)$ bit	$\Theta(1)$	$\Theta(L)$	elevato, poco stabile
PCSA	$\Theta(mL)$ bit	$\Theta(1)$	$\Theta(mL)$ o $\Theta(m)$	$\approx 0.78/\sqrt{m}$
LogLog	$\Theta(m \log(L - p + 2))$ bit	$\Theta(1)$	$\Theta(m)$	$\approx 1.30/\sqrt{m}$
HyperLogLog	$\Theta(m \log(L - p + 2))$ bit	$\Theta(1)$	$\Theta(m)$	$\approx 1.04/\sqrt{m}$
HyperLogLog++	$\Theta(k_{sp})$ o $\Theta(m)$ registri	$\Theta(1)$ ammortizzato	$\Theta(k_{sp})$ o $\Theta(m)$	$\Theta(1/\sqrt{m})$ con bias pratico ridotto

Tabella 3.4: Confronto di complessità e comportamento dell'errore per gli algoritmi principali

Le due tabelle vanno lette insieme. La prima mette a fuoco la forma dello stato, la seconda mostra come cambiano costo in memoria, tempi operativi ed errore al cambiare della struttura.

3.5 ALTRI PROBLEMI DI SKETCHING

Le due strutture che seguono non appartengono alla risoluzione del problema della conta degli elementi distinti, ma mostrano come lo sketching si estenda ad altri problemi. Si parte dal Bloom filter, nato per la membership approssimata, e si passa poi al Count-Min Sketch, che è diventato un riferimento classico per la stima di frequenze.

3.5.1 BLOOM FILTER

Def. 3.6 (Bloom filter). Un Bloom filter è un vettore di m bit con k funzioni di hash. L'inserimento di x imposta a uno le celle $B[h_1(x)], \dots, B[h_k(x)]$. Una query di appartenenza risponde “presente” se tutte le celle valgono uno.

Assumendo che le posizioni prodotte dalle funzioni di hash siano uniformi e si comportino come scelte indipendenti, la struttura non produce falsi negativi, ma può produrre falsi positivi con probabilità approssimativa [18, 19]:

$$p_{fp} \approx \left(1 - e^{-kn_{ins}/m}\right)^k$$

Algoritmo 3.6 Bloom filter

Choose k hash functions and a bit vector of length m

Initialize all bits to zero

for each insertion of element x

for $j = 1$ **to** k

 Set $B[h_j(x)] \leftarrow 1$

end for

end for

for each membership query on x

if $B[h_j(x)] = 1$ for every j

return “present”

else

return “absent”

end if

end for

ESEMPIO Consideriamo un Bloom filter con $m = 10$ bit e $k = 2$ funzioni di hash. Supponiamo $h_1(a) = 1$, $h_2(a) = 6$, $h_1(b) = 4$, $h_2(b) = 6$, $h_1(c) = 1$, $h_2(c) = 8$.

L'interrogazione su z produce un falso positivo, mentre l'interrogazione su q risponde in modo corretto con “assente”. L'esempio mostra il comportamento tipico della struttura.

Riguardo lo spazio computazionale della struttura dati, esso vale: m bit. L'aggiornamento e l'interrogazione costano $\Theta(k)$.

La combinazione di due strutture compatibili avviene tramite OR bit a bit.

Passo	Operazione	(h_1, h_2)	Celle toccate	Stato dei bit $[b_0 \dots b_9]$	Esito
1	inserisci(a)	$(1, 6)$	$b_1, b_6 \leftarrow 1$	$[0100001000]$	–
2	inserisci(b)	$(4, 6)$	$b_4, b_6 \leftarrow 1$	$[0100101000]$	–
3	inserisci(c)	$(1, 8)$	$b_1, b_8 \leftarrow 1$	$[0100101010]$	–
4	interroga(z)	$(1, 4)$	verifica b_1, b_4	$[0100101010]$	“presente”
5	interroga(q)	$(2, 9)$	verifica b_2, b_9	$[0100101010]$	“assente”

Tabella 3.5: Esempio operativo di Bloom filter

3.5.2 COUNT-MIN SKETCH

Tra gli sketch per frequenze, il Count-Min Sketch è una delle strutture che si sono imposte come riferimento classico nella letteratura sui flussi di dati.

Def. 3.7 (Count-Min Sketch). Il Count-Min Sketch [17] mantiene una matrice di contatori $C \in \mathbb{N}^{d \times w}$ e d funzioni di hash $h_1, \dots, h_d : \mathcal{U} \rightarrow [w]$. Per ogni aggiornamento dell'elemento x , si incrementa $C[j, h_j(x)]$ per ogni riga j . La stima puntuale della frequenza è:

$$\hat{f}(x) = \min_{j \in [d]} C[j, h_j(x)]$$

Assumendo funzioni di hash indipendenti tratte da una famiglia 2-universale, e ponendo:

$$w = \lceil e/\varepsilon \rceil \quad d = \lceil \ln(1/\delta) \rceil$$

vale con probabilità almeno $1 - \delta$:

$$f(x) \leq \hat{f}(x) \leq f(x) + \varepsilon \|f\|_1$$

Algoritmo 3.7 Count-Min Sketch

Choose d hash functions and a matrix C of size $d \times w$

Initialize all counters to zero

for each update of element x

for $j = 1$ **to** d

 Increment $C[j, h_j(x)]$

end for

end for

for each query on x

return $\min_{j \in [d]} C[j, h_j(x)]$

end for

ESEMPIO Consideriamo un Count-Min Sketch con $d = 2$ righe e $w = 5$ colonne. Supponiamo $h_1(a) = 1$, $h_2(a) = 3$, $h_1(b) = 4$, $h_2(b) = 1$, $h_1(c) = 1$, $h_2(c) = 4$ e processiamo il flusso a, b, a, c, a .

Passo	x	$(h_1(x), h_2(x))$	Aggiornamento	Stato riga 1	Stato riga 2
1	a	$(1, 3)$	$C[1, 1] ++, C[2, 3] ++$	$[0, 1, 0, 0, 0]$	$[0, 0, 0, 1, 0]$
2	b	$(4, 1)$	$C[1, 4] ++, C[2, 1] ++$	$[0, 1, 0, 0, 1]$	$[0, 1, 0, 1, 0]$
3	a	$(1, 3)$	$C[1, 1] ++, C[2, 3] ++$	$[0, 2, 0, 0, 1]$	$[0, 1, 0, 2, 0]$
4	c	$(1, 4)$	$C[1, 1] ++, C[2, 4] ++$	$[0, 3, 0, 0, 1]$	$[0, 1, 0, 2, 1]$
5	a	$(1, 3)$	$C[1, 1] ++, C[2, 3] ++$	$[0, 4, 0, 0, 1]$	$[0, 1, 0, 3, 1]$

Tabella 3.6: Esempio operativo di Count-Min Sketch

Dallo stato finale si ottiene

$$\hat{f}(a) = \min\{4, 3\} = 3$$

che in questo caso coincide con il valore esatto, pur restando la struttura in generale sovrastimante in presenza di collisioni.

La complessità in spazio è $\Theta(dw \log \|f\|_1)$ bit se si usano contatori interi standard. La complessità per aggiornare la struttura e fare query su di essa è $\Theta(d) = \Theta(\log(1/\delta))$.

La combinazione di due strutture compatibili avviene per somma componente per componente.

3.6 RELATED WORKS

I lavori richiamati nel capitolo si distribuiscono lungo tre linee principali. La prima è quella del calcolo approssimato degli elementi distinti, che parte da Probabilistic Counting [4], passa per LogLog [5], arriva a HyperLogLog [6] e trova in HyperLogLog++ una forma più adatta all'uso pratico su larga scala [7]. A questa linea si collegano anche lavori successivi che non cambiano il problema di riferimento, ma migliorano stimatori, costo in spazio o versatilità dello sketch, come CPC/FM85, SetSketch, HyperLogLogLog e UltraLogLog [22, 23, 24, 25].

La seconda linea riguarda l'uso distribuito degli sketch e la possibilità di combinare riassunti costruiti su partizioni diverse del flusso. In questa direzione rientrano i lavori su unioni di stream e sinossi mergeabili [26, 27, 16], insieme ai contributi che studiano in modo più specifico stime di unioni e intersezioni nella famiglia HLL [28, 21]. È soprattutto in questo secondo asse che si colloca il nucleo sperimentale della tesi: non tanto proporre un nuovo sketch, quanto osservare in quali condizioni l'unione di sketch resti semanticamente corretta, quando degrading e quando possa essere recuperata tramite normalizzazione.

La terza linea è quella delle strutture per problemi diversi dal solo count-distinct, come Bloom filter e Count-Min Sketch [18, 17]. Questi lavori non sono centrali rispetto agli esperimenti successivi, ma servono a mostrare che lo sketching non è limitato alla sola stima di F_0 e che il framework realizzato potrebbe essere esteso anche ad altre famiglie di algoritmi.

Nel complesso, il posizionamento della tesi è quindi intermedio tra stato dell'arte algoritmico e valutazione sperimentale. I capitoli successivi non si concentrano sulla progettazione di un nuovo stimatore, ma sull'imple-

mentazione uniforme di una parte rappresentativa di questa letteratura e sull'analisi del merge in casi omogenei ed eterogenei.

4

Implementazione

In seguito viene descritta l'implementazione del sistema sperimentale sviluppato per confrontare algoritmi di stima della cardinalità sui flussi di dati.

Lo studio degli algoritmi avviene in due modalità, una che simula la ricezione dei dati come in un servizio di streaming e l'altra che va a analizzare come le varie sketch prodotte dagli algoritmi si comportino alla loro unione.

Il capitolo è costruito attorno a tre nuclei principali:

- gli algoritmi, la loro interfaccia comune e le scelte adottate per la stesura in codice delle strutture descritte nella letteratura;
- le funzioni di hash, il loro ruolo nella semantica degli (*sketch*) e la ragione della loro selezione;
- l'infrastruttura di valutazione (*framework*), cioè il modo in cui dataset, algoritmi, metriche e serializzazione dei risultati sono stati composti in un'unica catena di esecuzione (*pipeline*) riproducibile.

I principali componenti del sistema sono stati implementati in C++, mentre in Python sono stato sviluppati gli script di generazione del dataset usato e di orchestrazione degli esperimenti.

4.1 OBIETTIVI PROGETTUALI

L'implementazione non nasce come semplice raccolta di algoritmi isolati, ma come piattaforma di confronto. Di conseguenza il progetto è stato guidato da quattro obiettivi principali.

Il primo obiettivo è uniformare algoritmi diversi sotto un contratto comune. Probabilistic Counting, LogLog, HyperLogLog e HyperLogLog++ condividono infatti la stessa interfaccia astratta, pur avendo stati interni e regole di stima molto differenti. Questa scelta permette al resto del sistema di invocare gli algoritmi senza conoscere i dettagli delle singole implementazioni.

Il secondo obiettivo è separare chiaramente la responsabilità dello sketch dalla responsabilità dell'hashing. In tutti gli algoritmi basati su registri, la funzione di hash non è un dettaglio accessorio, ma parte della semantica del calcolo. Per questo motivo l'hash non è codificato rigidamente dentro ciascun algoritmo, ma viene iniettato tramite un'interfaccia comune.

Il terzo obiettivo è rendere l'infrastruttura sperimentale riproducibile. Il seed, i parametri degli algoritmi, il dataset, la modalità di esecuzione e il percorso del file dei risultati devono essere tracciabili in modo deterministico e coerente tra esecuzione interattiva, orchestrazione batch e analisi finale.

Il quarto obiettivo è mantenere separati il livello algoritmico e il livello di misura. Il codice che aggiorna uno sketch non deve dipendere dal formato del dataset, dalla pianificazione dei punti di controllo (*checkpoint*) o dalla serializzazione in file CSV. Questa separazione riduce il rischio di introdurre dipendenze accidentali e rende l'infrastruttura estendibile a nuove famiglie di algoritmi e a nuovi esperimenti.

4.2 ARCHITETTURA GENERALE DEL SISTEMA

L'architettura complessiva è organizzata in tre strati:

- uno strato algoritmico, che contiene le implementazioni concrete degli sketch e il catalogo degli identificatori canonici;
- uno strato di supporto, che comprende dataset binario, funzioni di hash e tipi di configurazione;
- uno strato di coordinamento sperimentale, che comprende la riga di comando, il framework di valutazione, la raccolta delle metriche e la scrittura dei risultati.

Questa scomposizione corrisponde direttamente alla struttura del repository: `algorithms/` per gli sketch, `hashing/` per le funzioni di hash, `dataset/` per il formato binario, `simulation/` per il protocollo di valutazione e `cli/` per il coordinamento dell'esecuzione.

La Figura 4.1 riassume i moduli principali, il verso di utilizzo tra di essi e il rapporto con gli strumenti esterni in Python.

Dal punto di vista operativo, la catena di esecuzione completa è la seguente:

1. il dataset viene generato in Python e serializzato nel formato binario compresso usato dal framework;
2. la riga di comando carica il dataset, ne legge i metadati e costruisce il contesto di esecuzione;
3. il coordinatore di esecuzione seleziona gli algoritmi richiesti, istanzia la funzione di hash a tempo di esecuzione (*runtime*) e prepara i descrittori dell'esperimento;
4. il framework di valutazione percorre le partizioni del dataset e calcola le metriche in memoria;
5. il modulo di scrittura dei risultati serializza le statistiche in file CSV, che vengono poi elaborati nei notebook del capitolo sperimentale.

Questa architettura privilegia la separazione delle responsabilità. La riga di comando conosce i parametri della sessione, ma non la logica interna degli algoritmi. Gli algoritmi conoscono la propria logica di aggiornamento, ma non il formato del dataset. Il framework di valutazione conosce il protocollo sperimentale, ma non incorpora direttamente le politiche di generazione dei grafici o la struttura dei notebook.

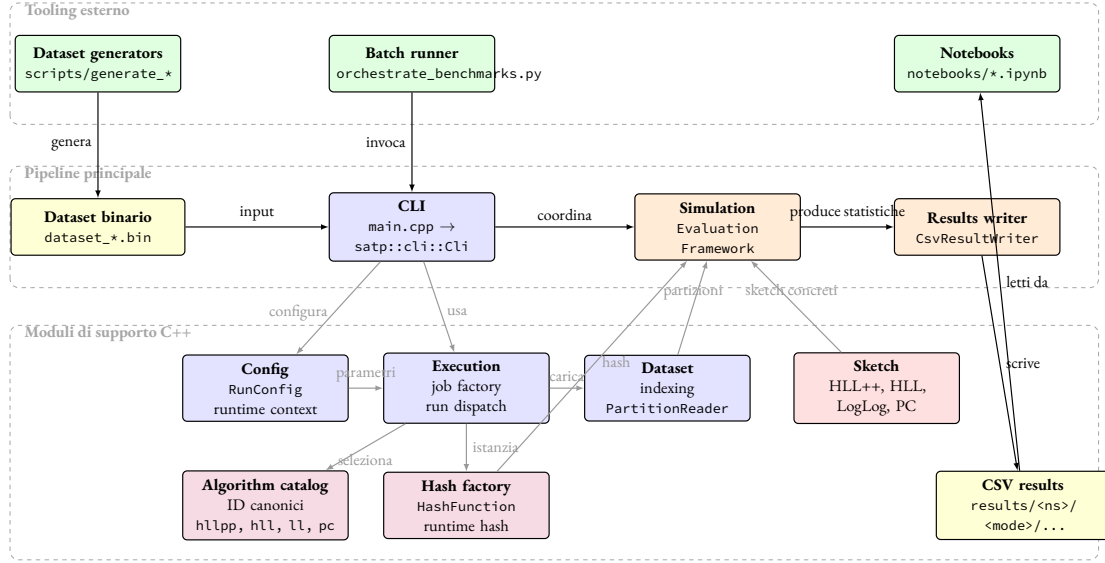


Figura 4.1: Architettura del sistema e principali dipendenze tra moduli e artefatti. Le frecce indicano le relazioni di dipendenza oppure il verso del flusso dei dati

4.3 ALGORITMI DI SKETCHING

Questa sezione raccoglie le scelte adottate per tradurre in codice gli algoritmi considerati nei capitoli precedenti. L'obiettivo è mostrare come siano stati resi confrontabili all'interno di un'unica infrastruttura, pur mantenendo le specificità delle singole implementazioni.

4.3.1 CRITERI DI SCELTA

Il nucleo algoritmico del progetto è dedicato al problema della stima del numero di elementi distinti. Le implementazioni presenti seguono una progressione storica e metodologica coerente con la letteratura studiata nei capitoli precedenti.

Prima dell'implementazione vera e propria degli algoritmi discussi precedentemente, è presente anche un'implementazione basica, che è stata inizialmente utilizzata come riferimento per gli altri algoritmi.

Essa è `NaiveCounting`. Questa classe non viene esposta come algoritmo degli esperimenti principali, ma è essenziale nei test del framework perché fornisce un riferimento privo di errore di stima.

Il primo algoritmo vero e proprio implementato è `ProbabilisticCounting`, che deriva dal lavoro di Flajolet e Martin [4]. Questa implementazione è interessante perché rappresenta il primo punto della linea evolutiva degli algoritmi probabilistici per il conteggio degli elementi distinti.

Vengono poi a seguire `LogLog` e `HyperLogLog`, che discendono rispettivamente dai lavori di Durand e Flajolet [5] e di Flajolet, Fusy, Gandouet e Meunier [6]. Questi due algoritmi condividono la struttura a registri e consentono di studiare in modo diretto come varia la qualità della stima a parità di idea architettonica di base.

L'ultima implementazione è `HyperLogLog++`, che segue la variante pratica proposta da Heule, Nunkesser e Hall [7]. È l'algoritmo più importante della base di codice perché rappresenta sia il riferimento per i confronti in modalità di flusso sia il punto di partenza per lo studio del comportamento dell'unione tra sketch in casi non ottimali.

La selezione degli algoritmi non è quindi casuale. Essa copre:

- un riferimento esatto;
- un primo algoritmo probabilistico compatto;
- due algoritmi a registri della famiglia `LogLog` e `HyperLogLog`;
- la variante più pratica e completa della stessa linea evolutiva.

Dal punto di vista del catalogo applicativo, gli esperimenti pubblici espongono quattro identificatori canonici: `hllpp`, `hll`, `ll`, `pc`. Nel file `AlgorithmCatalog.cpp` è presente anche l'identificatore `naive` per il riferimento interno, ma questo non fa parte dell'elenco pubblico degli algoritmi selezionabili dalla riga di comando. In modo coerente, `getIdsOfSupportedAlgorithms()` espone solo i quattro identificatori pubblici e non include `naive`.

4.3.2 INTERFACCIA COMUNE

L'interfaccia astratta comune è definita in `Algorithm.h`. Ogni algoritmo riceve nel costruttore un riferimento costante a una funzione di hash e deve implementare i seguenti metodi:

- `process(uint32_t id)`, che aggiorna lo sketch con un nuovo valore;
- `count()`, che restituisce il conteggio esatto o la stima corrente;
- `merge(const Algorithm& other)`, che combina due stati compatibili;
- `reset()`, che azzerava lo stato interno;
- `getName()`, che restituisce il nome canonico usato nel resto del sistema.

Questa interfaccia esprime due scelte progettuali precise. La prima è che la funzione di hash appartenga al contesto dell'algoritmo e non sia lasciata come responsabilità del chiamante a ogni aggiornamento. La seconda è che il merge sia parte dell'interfaccia di primo livello e non un'estensione opzionale aggiunta solo ad alcuni algoritmi.

Nel framework esiste anche un controllo di concetto a livello di compilazione, definito in `SketchFactory.h`, che impedisce di usare nelle modalità di merge algoritmi privi dell'operazione di unione richiesta.

4.3.3 SCELTE PROGETTUALI COMUNI

Pur seguendo lavori diversi, le implementazioni condividono alcune convenzioni.

La prima convenzione è la validazione esplicita dei parametri nel costruttore. Ogni algoritmo ha dei limiti provenienti dal loro articolo in cui sono stati definiti, nei parametri che utilizza come, ad esempio, la dimensione dei registri o la lunghezza della bitmap.

I limiti degli algoritmi non sono lasciati a vincoli impliciti, ma producono eccezioni `invalid_argument` se la configurazione non appartiene al dominio supportato.

La seconda convenzione è la distinzione tra interfaccia astratta e overload tipizzato del merge. Ogni classe implementa `merge(const Algorithm&)` con un controllo dinamico del tipo e poi delega a una variante tipizzata `merge(const TipoConcreto&)`. In questo modo il contratto comune resta stabile, ma l'implementazione concreta può esprimere in modo chiaro i controlli di compatibilità parametrica.

La terza convenzione è l'uso di identificatori canonici separati dal nome stampato. Il catalogo algoritmico centralizza infatti la traduzione tra identificatori brevi, usati dalla riga di comando e dai percorsi dei risultati e nomi completi, usati nei file CSV e nella tesi.

La quarta convenzione è la separazione tra stato interno dello sketch e logica di serializzazione dei parametri. Il formato testuale dei parametri $k = \dots, L = \dots$ o combinazioni analoghe non è costruito dentro le classi degli algoritmi, ma nel livello di coordinamento delle unità di esecuzione (*job*). Questo rende più semplice mantenere coerenti descrizione testuale, nomi dei file dei risultati e ricostruzione degli esperimenti.

4.3.4 IMPLEMENTAZIONE DEI SINGOLI ALGORITMI

A seguire vengono descritte le implementazioni degli algoritmi precedentemente descritti.

ALGORITMO ESATTO `NaiveCounting` mantiene un insieme ordinato di identificatori distinti tramite `uint32`. Il metodo `process` inserisce un elemento solo se non è già presente, il metodo `count` restituisce la cardinalità dell'insieme e il merge esegue un'unione insiemistica. Questa classe riceve comunque una funzione di hash, per uniformità di interfaccia, ma non la usa nella logica interna.

PROBABILISTIC COUNTING L'implementazione di `ProbabilisticCounting` segue l'idea originale di Flajolet e Martin [4]. Lo stato è rappresentato da una bitmap memorizzata in un intero a 32 bit, con il vincolo applicativo $L \in [1, 31]$ e l'aggiornamento usa `hash32`, tronca il risultato a L bit, individua la posizione del primo uno a partire dal bit meno significativo con `count_r_zero` e attiva il bit corrispondente. La stima usa invece `count_r_one` per ricavare l'indice del primo zero e applica la costante di correzione ϕ del metodo originario. Il merge usa l'operatore OR bit per bit e richiede uguaglianza del parametro L nei due sketch.

LOGLOG L'implementazione usa un vettore di registri `uint8_t` e mantiene in modo incrementale la somma dei registri nella quantità $\sum_j M[j]$ aggiornata a ogni inserimento. Questa scelta evita di ricalcolare l'intera somma a ogni chiamata di `count`. Il costruttore impone un'implementazione aderente al dominio usato nel lavoro di riferimento. Nel dominio implementato, con $k \in [4, 16]$ e $L = 32$ l'aggiornamento estrae i primi k bit dell'hash a 32 bit per selezionare il registro e usa il resto dei bit per calcolare il rango ρ e il merge applica il massimo componente per componente e poi ricostruisce la somma dei registri aggiornata.

HYPERLOGLOG `HyperLogLog` mantiene ancora un vettore di registri `uint8_t`, ma a differenza di `LogLog` memorizza anche due quantità aggregate che servono direttamente alla stima: la somma delle potenze inverse $\sum_j 2^{-M[j]}$ e il numero di registri nulli. Questa scelta rende più efficiente la valutazione dei diversi regimi di stima. Anche qui il dominio è ristretto al caso classico. Nel dominio implementato, con $k \in [4, 16]$ e $L = 32$ il metodo `count` implementa tre regimi: correzione per cardinalità piccole tramite *linear counting*, stima armonica

nel regime intermedio e correzione logaritmica nel regime grande, in accordo con [6]. Il merge richiede uguaglianza dei parametri k e L oltre al numero di bucket e poi ricalcola le grandezze aggregate sullo stato risultante.

HYPERLOGLOG++ HyperLogLog++ è l'implementazione più articolata della base di codice. Il parametro costruttivo è memorizzato come p ma nel resto del framework viene serializzato uniformemente come k per mantenere un formato dei parametri coerente tra famiglie algoritmiche. Nel dominio supportato con $p \in [4, 18]$ questa implementazione, a differenza di HyperLogLog, usa sempre l'hash a 64 bit.

Lo stato interno alterna due rappresentazioni:

- una rappresentazione sparsa, basata su un insieme temporaneo `tmpSet` e su una lista compressa `sparseList`;
- una rappresentazione densa, basata su un vettore di registri `registers`.

La transizione dalla forma sparsa alla forma densa avviene quando il costo in bit della rappresentazione sparsa supera il costo della rappresentazione densa. Questa scelta è implementata confrontando `compressedSparseBits()` e `denseBits()`. La stima finale integra inoltre due componenti aggiuntive: una tabella di bias e una soglia per la scelta tra *linear counting* e stima corretta, entrambe mantenute in `hllpp_tables.cpp`.

Il merge diretto è definito solo quando il parametro p coincide nei due sketch. Se entrambi gli sketch sono ancora in formato sparso, il merge procede fondendo le codifiche compresse; se almeno uno dei due è già in formato denso, entrambi vengono prima portati in forma densa e poi fusi registro per registro. La classe espone anche due operazioni aggiuntive, `reducedTo(...)` e `reducedToNaive(...)`, usate nel framework di merge eterogeneo per studiare il caso di mismatch di precisione. Nei casi `recoverable` di mismatch di precisione, il framework costruisce sia il riferimento seriale sia il riferimento omogeneo alla precisione minima $\min(k_{left}, k_{right})$ in modo coerente per entrambi i confronti.

4.4 FUNZIONI DI HASH

Le funzioni di hash costituiscono un livello trasversale dell'intera implementazione, perché incidono sia sul funzionamento degli sketch sia sulla correttezza dei confronti sperimentali. Per questo motivo vengono discusse a parte, mettendo in evidenza il loro ruolo, l'interfaccia comune e i criteri di selezione adottati.

4.4.1 RUOLO DELL'HASH NEGLI SKETCH

Negli algoritmi implementati l'hash non serve solo a ottenere un valore di hash uniforme. Esso è parte integrante della funzionalità degli algoritmi, per due principali ragioni: per questioni di scegliere in maniera libera quale funzione di hash utilizzare e per poter sperimentare il merge eterogeneo in caso di funzioni di hash differenti.

In `ProbabilisticCounting`, i bit dell'hash determinano la posizione del primo uno osservato nella bitmap. In `LogLog` e `HyperLogLog`, i primi k bit identificano il registro, mentre i bit restanti definiscono un rango indicato con ρ e nel caso di `HyperLogLog++` la stessa idea viene estesa al caso a 64 bit e viene riutilizzata sia nella rappresentazione densa sia in quella sparsa.

Da questa osservazione discende una conseguenza importante per tutta la tesi: il processo di merge corretto non dipende soltanto dal tipo dell'algoritmo, ma anche dalla semantica condivisa dell'hash. Due sketch costruiti con hash o seed diversi non stanno necessariamente rappresentando gli stessi sottoflussi logici.

4.4.2 INTERFACCIA COMUNE DELLE HASH

L'astrazione comune delle funzioni di hash è definita in `HashFunction.h`. L'interfaccia espone tre operazioni:

- `hash64(uint64_t)`, che rappresenta l'operazione fondamentale;
- `hash32(uint64_t)`, che per default proietta il risultato a 64 bit sui 32 bit più significativi;
- `name()`, che restituisce il nome canonico della funzione.

Questa interfaccia permette di iniettare una funzione di hash in qualunque algoritmo senza cambiare il contratto della classe `Algorithm`. Il fatto che `hash32` sia definito come proiezione standard della versione a 64 bit consente inoltre di mantenere un'unica gerarchia di hash anche quando algoritmi diversi operano su ampiezze di parola diverse.

4.4.3 FUNZIONI SCELTE E MOTIVAZIONE

La selezione delle funzioni di hash è centralizzata in `HashFactory.cpp`. Il sistema, attualmente, supporta quattro funzioni di hash: `splitmix64`, `xxhash64`, `murmurhash3`, `siphash24`.

Questa scelta risponde a un criterio di copertura del dominio sperimentale, piuttosto che alla ricerca di una singola funzione ottima in assoluto.

`splitmix64` è la funzione di default del sistema. È semplice da istanziare, non richiede stato esterno nel costruttore corrente e fornisce un riferimento leggero e deterministico per tutti gli esperimenti.

`xxhash64` rappresenta una funzione non crittografica molto diffusa nelle applicazioni pratiche, con supporto esplicito al seed nella classe `XXHash64`. La sua presenza è utile perché combina semplicità implementativa e uso realistico nei sistemi applicativi.

`murmurhash3` rappresenta un secondo riferimento pratico molto diffuso. Nel codice viene usata una variante ridotta a un blocco da 64 bit, seedata e adatta al caso in cui gli identificatori siano interi a 32 bit promossi a 64 bit.

`siphash24` completa il quadro con una funzione della famiglia `SipHash`. Nel codice attuale viene istanziata con le chiavi di default della classe `SipHash24`. La sua presenza è utile perché aggiunge al confronto una funzione con una costruzione diversa da quelle puramente non crittografiche più comuni.

L'obiettivo della selezione non è quindi confrontare soltanto velocità diverse, ma rendere possibile una matrice sperimentale in cui la semantica dell'hash può essere mantenuta costante oppure modificata in modo controllato.

4.4.4 SEED E RIPRODUCIBILITÀ

Il seed entra nel sistema in due punti distinti. Il primo luogo in cui viene definito è il seed del dataset, che viene memorizzato nell'header binario e propagato nel contesto durante il tempo di esecuzione. Il secondo è il seed locale

della funzione di hash, che nel caso del merge eterogeneo può essere impostato separatamente per lato sinistro e destro.

La factory `getHashFunctionBy(...)` applica le seguenti regole:

- se non viene specificato alcun nome, viene istanziata `SplitMix64`;
- se viene specificato un nome, la factory richiede anche un seed;
- `XXHash64` e `MurmurHash3` usano esplicitamente il seed passato;
- `SplitMix64` ignora il seed a livello della classe concreta;
- `SipHash24` è istanziata con le chiavi di default previste dalla classe concreta, quindi il seed configurato nella sessione non agisce come leva uniforme su questa hash.

Nel flusso standard della riga di comando, il seed usato per la funzione di hash ha come valore di default il seed del dataset. Nel merge eterogeneo, invece, i campi `leftHashSeed` e `rightHashSeed` prevalgono quando sono valorizzati, mentre in assenza di override resta attivo il valore di default al seed del dataset.

4.5 FRAMEWORK SPERIMENTALE

Dopo avere definito algoritmi e hashing, resta da descrivere il livello che coordina l'esecuzione degli esperimenti. In questa sezione vengono quindi presentati i requisiti del framework, la sua organizzazione interna e i flussi con cui vengono prodotti i risultati.

4.5.1 REQUISITI DEL FRAMEWORK

L'infrastruttura sperimentale doveva soddisfare contemporaneamente requisiti diversi.

Dal lato algoritmico, doveva essere possibile usare lo stesso schema di esecuzione per sketch differenti senza duplicare codice di attraversamento del dataset.

Essenzialmente, si voleva rendere disaccoppiati il framework e la parte algoritmica/hashing.

Deve essere possibile aggiungere algoritmi o funzioni di hashing con il minimo cambiamento all'interno del framework.

Dal lato metodologico, dovevano essere supportate tre modalità diverse:

- una modalità di flusso, per osservare l'evoluzione della stima lungo i prefissi;
- una modalità di merge omogenea, per confrontare merge e riferimento seriale;
- una modalità di merge eterogenea, per descrivere casi validi, recuperabili o non validi e misurarne il comportamento.

La distinzione tra le due modalità di merge era necessaria, per verificare che l'intero sistema si comportasse in maniera accurata nel caso di merge con parametri sbagliati.

Inizialmente, non si voleva far accadere erroneamente che ci potessero essere dei merge tra algoritmi diversi o con parametri diversi.

Dal lato operativo, il sistema doveva derivare automaticamente dai metadati del dataset il numero di partizioni, la dimensione della partizione e il seed, così da evitare configurazioni incoerenti tra dataset e sessione di esecuzione.

Uno dei punti più importanti era che il seed fosse uniforme in tutto il processo, a partire dalla generazione del dataset fino all'ottenimento dei risultati.

4.5.2 STRUTTURA DEL FRAMEWORK

La struttura pubblica è organizzata in tre moduli principali:

- `cli` come interfaccia interattiva verso l'utente;
- il modulo pubblico del dataset, esposto da `Dataset.h`, come punto di accesso all'indicizzazione e al caricamento delle partizioni;
- `EvaluationFramework`, riesportato da `Simulation.h`, come coordinatore delle modalità di valutazione.

La classe `EvaluationFramework` contiene un indice del dataset binario, un oggetto `EvaluationMetadata` derivato dai metadati del dataset e una funzione di hash posseduta tramite `unique_ptr`. Da questa struttura discende una proprietà importante: il framework non ha bisogno di rileggere la configurazione globale a ogni esecuzione, perché il suo contesto fondamentale è già materializzato all'atto della costruzione.

Nel flusso eterogeneo, però, la funzione di hash posseduta da `EvaluationFramework` non governa direttamente i due lati del confronto. In `HeterogeneousMergeEvaluation.tpp`, per ogni coppia vengono istanziate hash dedicate a partire dal descrittore: hash sinistra, hash destra, hash del riferimento seriale e hash del riferimento omogeneo quando presente. In termini operativi, le istanze vengono costruite per ogni coppia con `getHashFunctionBy(...)` usando i campi `left`, `right`, `serialReference` e `homogeneousBaseline` del descrittore.

Il livello `cli` non implementa direttamente gli algoritmi. Esso costruisce piuttosto oggetti del tipo `AlgorithmJob` tramite `JobFactory.cpp`, uno per ogni combinazione di algoritmo, parametri, hash e modalità richiesta. Questo permette di tenere separata la logica di configurazione dalla logica di esecuzione.

La configurazione della sessione vive in `RunConfig`, mentre i metadati derivati dal dataset vivono in `DatasetRuntimeContext`. Questa separazione è importante: parametri come `k`, `l`, `hashFunction` o `mergeStrategy` appartengono alla sessione corrente, mentre `sampleSize`, `runs` e `seed` vengono sempre letti dal file binario e non possono divergere dal dataset reale.

4.5.3 FLUSSI DI ESECUZIONE

L'infrastruttura espone tre metodi principali:

- `evaluateStreaming(...)`
- `evaluateMergePairs(...)`
- `evaluateHeterogeneousMergePairs(...)`

Nel flusso di tipo `streaming`, il framework percorre ogni partizione elemento per elemento e raccoglie statistiche ai punti di controllo fissati dal pianificatore dei punti di controllo (*planner*) La verità di prefisso viene ricostruita dal bitset delle prime occorrenze memorizzato nel dataset.

Nel flusso di tipo `merge`, le partizioni vengono accoppiate come $(0, 1), (2, 3), \dots$ e per ogni coppia si costruiscono due sketch separati, se ne calcola il merge e si confronta il risultato con uno sketch seriale ottenuto processando gli stessi dati in sequenza.

Nel flusso di tipo `merge_heterogeneous`, le due partizioni della coppia possono essere elaborate con contesti locali distinti per hash, seed e parametri. Il descrittore `HeterogeneousMergeRunDescriptor` definisce:

- il contesto locale sinistro;
- il contesto locale destro;
- la strategia operativa del merge;
- la validità semantica del caso;
- la topologia sperimentale;
- un eventuale riferimento seriale;
- un eventuale riferimento omogeneo.

Questa struttura rende possibile studiare nello stesso framework sia casi omogenei sia casi di incompatibilità o di recuperabilità controllata. Nel caso `recoverable` con `HyperLogLog++`, quando il mismatch è solo di precisione, il descrittore porta il riferimento seriale e il riferimento omogeneo alla precisione minima $\min(k_{left}, k_{right})$ in modo che entrambi i confronti usino la stessa precisione comune.

4.5.4 DESCRITTORI DI ESECUZIONE E FORMATO DEI RISULTATI

L'infrastruttura non restituisce direttamente file, ma vettori di punti statistici in memoria. La scrittura persistente è affidata a `CsvResultWriter`, che espone tre funzioni principali:

- `appendStreaming(...)`
- `appendMergePairs(...)`
- `appendHeterogeneousMergePairs(...)`

Questa scelta evita di accoppiare la logica di misura al dettaglio della serializzazione.

I percorsi dei risultati sono costruiti da `buildResultCsvPath(...)` secondo lo schema generale

```
results/<namespace>/<mode>/<algorithm>/<hash>/<params>/results_*.csv
```

Il livello hash nel percorso è importante perché rende distinguibili esperimenti che usano lo stesso algoritmo e gli stessi parametri, ma funzioni di hash diverse. Nel caso del merge eterogeneo il nome del livello hash non è una singola funzione, ma una stringa composta che descrive entrambi i lati del confronto.

Lo schema CSV del merge eterogeneo è più esplicito dei CSV classici:

- non ha il campo di primo livello `params`;
- non usa il vecchio campo `seed`;
- usa `dataset_seed` e i campi separati `left_*/right_*`.

Nella forma completa, il record separa esplicitamente:

- `dataset_seed`;
- `left_hash`, `right_hash`, `left_hash_seed`, `right_hash_seed`;
- `left_params`, `right_params`;
- strategia di fusione;
- classificazione di validità;
- topologia;
- riferimento esatto, seriale e riferimento omogeneo;
- stima da fusione, stima seriale e scostamento rispetto al riferimento omogeneo.

Questa decisione è stata presa con l'obiettivo di non registrare solamente l'unione di due sketch ma è necessaria per riprodurre correttamente quale fosse il contesto dell'unione.

4.5.5 MOTIVAZIONI ARCHITETTURALI

Le scelte architetturali principali possono essere riassunte in quattro punti.

La prima è l'iniezione della funzione di hash a tempo di esecuzione. Questa decisione evita di duplicare il codice degli algoritmi e rende possibile confrontare famiglie di hash diverse senza cambiare le classi degli sketch.

La seconda è la derivazione di `runs`, `sample_size` e `seed` direttamente dal dataset. In questo modo i metadati del dataset sono l'unica sorgente autorevole della configurazione sperimentale.

La terza è la separazione tra valutazione e serializzazione. Le modalità del framework producono strutture dati in memoria; lo strato architetturale della parte da riga di comando si occupa poi di trasformarle in file persistenti.

La quarta è l'uso di unità di esecuzione descrittive. Invece di incorporare grandi blocchi condizionali dentro la riga di comando, il sistema costruisce una lista di `AlgorithmJob`, ciascuno con specifica, parametri e funzione di esecuzione. Questa scelta riduce l'accoppiamento tra configurazione e logica operativa.

4.6 DATASET E RIPRODUCIBILITÀ

Il sistema usa un solo protocollo di generazione dei dati, chiamato `prefix_constant_rho`.

In questa sezione non entreremo in dettaglio, poiché verrà trattato nel capitolo 5. Ma resta necessaria perché la misurazione dei risultati dipende direttamente da alcune proprietà del dataset, in particolare dalla struttura in cui è caratterizzato il dataset.

4.6.1 FORMATO BINARIO E CARICAMENTO PER PARTIZIONE

Il dataset è serializzato in un formato binario compresso, identificato da `MAGIC=SATPDBN2` e `VERSION=2`. L'header globale memorizza:

- il numero di elementi per partizione indicato con n nel file;
- il numero di distinti per partizione indicato con d nel file;
- il numero di partizioni indicato con p nel file;
- il seed globale del dataset.

L'header è seguito da una tabella delle partizioni. Ogni entry contiene offset, dimensioni compresse dei payload, cardinalità locale e codifiche dei blocchi. Valori e bitset di verità sono compressi separatamente con `zlib`. Nel formato corrente ogni entry della tabella ha dimensione 60 byte e contiene i campi `values_offset`, `values_byte_size`, `truth_offset`, `truth_byte_size`, i valori locali di n e d più le codifiche `ENCODING_ZLIB_U32_LE` e `ENCODING_ZLIB_BITSET_LE`.

Campo header	Significato
<code>MAGIC=SATPDBN2</code>	Identificatore del formato
<code>VERSION=2</code>	Versione del file binario
n	Elementi per partizione
d	Distinti per partizione
p	Numero di partizioni
seed	Seed globale del dataset

Tabella 4.1: Campi principali dell'header del dataset binario

La scelta progettuale importante è che il framework carica in memoria una sola partizione alla volta tramite `PartitionReader`. Il costo in memoria è quindi proporzionale alla partizione corrente e non all'intero dataset.

4.6.2 BITSET DI VERITÀ E CARDINALITÀ DI PREFISSO

Ogni partizione contiene, oltre al vettore dei valori, un bitset di verità in cui il bit b_t vale 1 se l'elemento in posizione t è una prima occorrenza nella partizione. La cardinalità reale di prefisso viene quindi ricostruita nel framework con

$$F_0(t) = \sum_{i=1}^t b_i$$

Questa informazione rende possibile la valutazione in modalità di flusso senza dover ricostruire ogni volta un insieme esplicito dei valori distinti visti fino al punto corrente.

4.6.3 PROTOCOLLO `prefix_constant_rho`

I dataset utilizzati sono stati generati con uno script `generate_prefix_constant_rho_dataset_bin.py`. Il generatore impone il vincolo che:

$$n \bmod d = 0$$

In questo modo il rapporto degli elementi totali rispetto al numero di elementi distinti:

$$\rho = \frac{n}{d}$$

resta costante in ogni partizione. La costruzione di ciascuna partizione segue quattro passi:

1. viene derivato un seed locale deterministico a partire dal seed globale e dall'indice della partizione;
2. vengono estratti d identificatori distinti nel dominio $[0, \min(n, 2^{32}) - 1]$ come valori `uint32`;
3. all'inizio di ogni blocco di ampiezza ρ viene emessa una nuova prima occorrenza;
4. le restanti posizioni del blocco sono riempite campionando uniformemente tra gli identificatori già osservati.

Poiché ogni blocco di lunghezza ρ introduce esattamente una nuova prima occorrenza, al termine del j -esimo blocco il prefisso contiene esattamente j elementi distinti:

$$F_0(j \cdot \rho) = j$$

Ricordiamo che $F_0(t)$ è il numero di elementi distinti osservati nel prefisso di lunghezza t .

Il protocollo separa quindi in modo controllato la crescita della cardinalità reale dal numero medio di ripetizioni. Questa proprietà è ciò che rende leggibili e confrontabili i grafici del capitolo sperimentale.

4.6.4 RIPRODUCIBILITÀ DEL DATASET

La riproducibilità del dataset è garantita da tre elementi:

- il seed globale è descritto nell'header del file;
- ogni partizione usa un seed derivato in modo deterministico tramite una funzione nel framework;
- il generatore è testato esplicitamente con dei test di unità in `test_generate_prefix_constant_rho_dataset.py`.

I test verificano la correttezza del formato binario, oltre alla proprietà $\sum_t b_t = d$ e alla relazione $F_0(j \cdot \rho) = j$ insieme alla presenza di una sola nuova prima occorrenza per blocco e al determinismo del file prodotto.

4.7 VALIDAZIONE

La validazione del sistema è distribuita tra test algoritmici, test del framework e test del generatore del dataset.

4.7.1 TEST SUGLI ALGORITMI

In `tests/algorithms` sono presenti test che coprono:

- l'accuratezza di base degli algoritmi sui dataset di riferimento;
- la validazione dei domini parametrici per HyperLogLog, HyperLogLog++ e LogLog;
- le proprietà del merge, in particolare commutatività, idempotenza e correttezza rispetto al caso omogeneo;
- la compatibilità parametrica del merge, con eccezioni esplicite sui mismatch non ammessi.

4.7.2 TEST DEL FRAMEWORK

Il file `simulation/EvaluationFrameworkTest.cpp` controlla che:

- i metadati del framework siano derivati dal dataset e non da valori impostati manualmente;
- in modalità di flusso il riferimento esatto produca sempre $\hat{F}_0(t) = F_0(t)$ per ogni punto di controllo;
- il pianificatore dei punti di controllo copra correttamente il flusso e rispetti la configurazione, cioè che avvengano esattamente un certo numero di checkpoint;
- il merge del riferimento esatto coincida con il riferimento seriale;
- la serializzazione dei file CSV sia coerente con gli header previsti;
- il merge eterogeneo copra i casi `valid`, `recoverable` e `invalid` con le relative strategie.

In particolare, i test sul merge eterogeneo verificano quattro regimi:

- caso `valid` con strategia `direct`;
- caso `invalid` con strategia `reject`;
- caso `recoverable` con strategia `reduce_then_merge`;
- caso `recoverable` con strategia `unsafe_naive_merge`.

4.7.3 TEST DEL GENERATORE E DELLA CATENA DI ESECUZIONE

La componente Python è validata da test dedicati sui generatori. In aggiunta, gli script di orchestrazione costruiscono i valori dei comandi per la riga di comando in modo deterministico, fissando il nome dei file risultanti e del loro percorso, funzioni di hash e griglie parametriche. Anche questa scelta contribuisce alla riproducibilità complessiva della catena di esecuzione.

5

Risultati sperimentali

I risultati sperimentali raccolti seguono la stessa progressione con cui è stato costruito il lavoro. Si parte dalla verifica del contesto sperimentale, cioè dal controllo empirico del dataset `prefix_constant_rho` e dalla scelta di un seed stabile per gli esperimenti successivi. Su questa base vengono poi confrontati gli algoritmi, osservando come evolve la stima lungo i prefissi e come essa cambi al variare del rapporto tra elementi processati e distinti e del parametro di precisione.

La seconda parte è dedicata al merge di sketch. Il punto di partenza è il caso omogeneo, usato come controllo di correttezza dell'infrastruttura. Si passa poi ai casi eterogenei, distinguendo tra situazioni `valid`, `invalid` e `recoverable`. Gli esperimenti su funzioni di hash diverse e su mismatch di precisione servono da un lato a verificare quanto emerso nella letteratura, dall'altro a mostrare che il framework costruito consente di studiare in modo sistematico casi di merge non ideali.

AMBIENTE DI TEST I risultati sono stati ottenuti su una workstation con processore AMD Ryzen 7 9700X, 32 GB di memoria DDR5 e unità a stato solido NVMe da 1 TB, con sistema operativo CachyOS e ambiente grafico COSMIC.

La catena di compilazione utilizzata è:

- CMake 4.3.0;
- Clang 22.1.1;
- target `x86_64-pc-linux-gnu`.

5.1 VALIDAZIONE DEL CONTESTO SPERIMENTALE

Prima di confrontare gli algoritmi è necessario verificare che il contesto di prova sia coerente con le ipotesi alla base degli esperimenti successivi. La sezione raccoglie quindi i controlli preliminari sul dataset utilizzato e sulla scelta del

seed mantenuto nelle esecuzioni principali.

5.1.1 VERIFICA EMPIRICA DEL DATASET A PREFISSO COSTANTE

Gli esperimenti usano il dataset a prefisso costante con le seguenti configurazioni:

$$n = 10^7, \quad p = 50, \quad seed = 21041998, \quad d \in \{10^5, 2 \cdot 10^5, 5 \cdot 10^5, 10^6, 2 \cdot 10^6, 5 \cdot 10^6, 10^7\}$$

$$\rho = \frac{n}{d} \in \{100, 50, 20, 10, 5, 2, 1\}$$

I sette valori di ρ corrispondono ai rapporti tra elementi processati e distinti usati nel resto del capitolo.

La validazione empirica del comportamento del dataset è mostrata nelle Figure 5.1 e 5.2. La prima figura mostra il rapporto $\rho_t = t/F_0(t)$ sui punti di controllo effettivamente usati nella modalità di flusso, mentre la seconda isola i soli punti di bordo dei blocchi, nei quali il rapporto assume il valore costante previsto dalla costruzione.

Dalle figure si osserva che il comportamento atteso è rispettato. All'interno di ciascun blocco la cardinalità distinta non cresce, perché non vengono introdotte nuove prime occorrenze, mentre ai bordi di blocco essa aumenta di una sola unità. Ne segue che il rapporto $\rho_t = t/F_0(t)$ assume il valore teorico ρ esattamente nei punti $t = j \cdot \rho$, in accordo con la costruzione del dataset.

5.1.2 SCELTA DEL SEED

Una verifica preliminare, svolta nelle prime iterazioni degli esperimenti, ha indicato che l'effetto del seed resta ininfluenza rispetto alle differenze tra algoritmi e tra le varie modalità di merge.

Non sono stati fatti confronti con più dataset con seed diversi ma stessi parametri anche per una questione di spazio richiesto.

Ogni dataset pesa nell'ordine dei 2 GB dopo la compressione, per 50 milioni di elementi. Le configurazioni testate sono tante e sarebbe diventato arduo testarle tutte.

Per questo motivo gli esperimenti principali fissano un seed unico pari a 21041998.

5.2 ESPERIMENTI SUGLI ALGORITMI

Gli esperimenti sugli algoritmi sono organizzati in due gruppi complementari. Il primo ha lo scopo di osservare come cambia la qualità della stima al variare della cardinalità reale del prefisso e del rapporto tra elementi processati e distinti. L'interesse non è limitato al solo errore medio, ma comprende anche la dispersione della stima, così da distinguere chiaramente tra fenomeni di bias e fenomeni di scarsa robustezza.

Per isolare questi effetti sono state considerate due viste del problema. Nella prima si fissa il rapporto

$$\rho = \frac{n}{d}$$

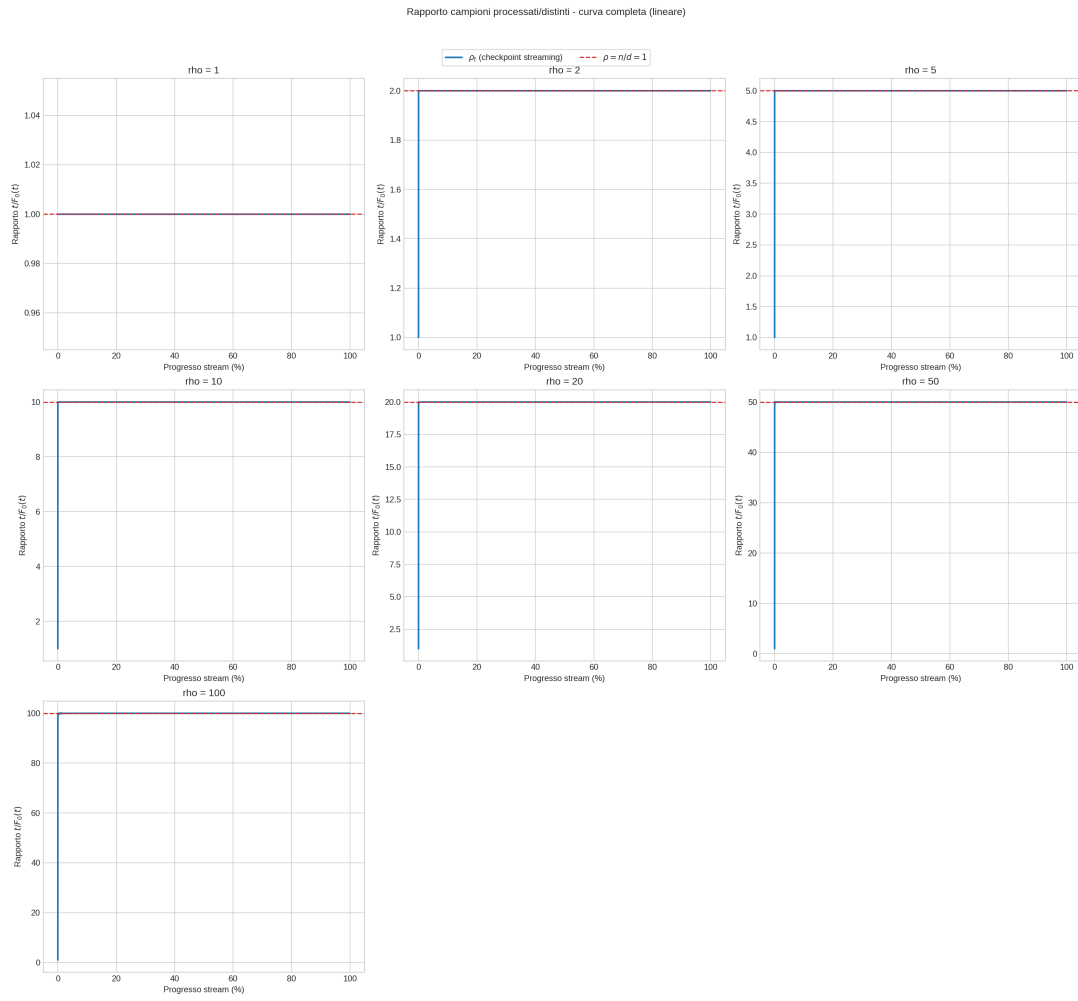


Figura 5.1: Rapporto $\rho_t = t/F_0(t)$ osservato ai checkpoint del dataset

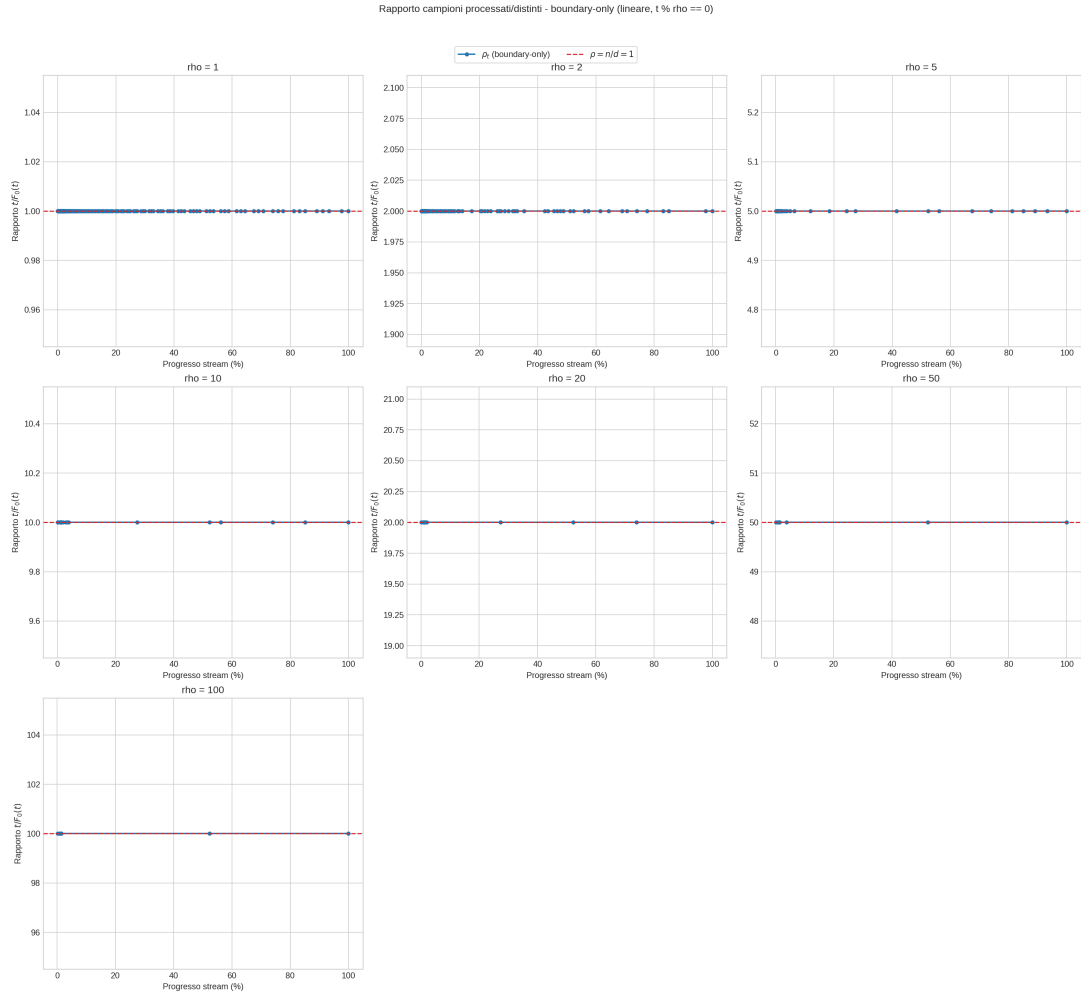


Figura 5.2: Rapporto ρ_t osservato ai bordi di blocco

e si osserva la stima in funzione della cardinalità reale del prefisso. Nella seconda si fissano valori obiettivo della cardinalità reale e si studia la stima in funzione del rapporto medio di ripetizione del prefisso,

$$\rho_t = \frac{t}{F_0(t)}$$

in modo da separare, per quanto possibile, l'effetto della crescita degli elementi distinti da quello delle ripetizioni medie.

Il secondo gruppo di esperimenti mantiene fisso il contesto di esecuzione e varia invece il parametro interno di precisione di ciascun algoritmo. Questa analisi serve a capire non solo quale algoritmo sia più accurato nel complesso, ma anche quanto il suo comportamento dipenda dalla scelta del parametro e se il miglioramento ottenuto all'aumentare delle risorse sia regolare oppure no.

5.2.1 CONFIGURAZIONE UTILIZZATA

Gli esperimenti usano il dataset:

```
datasets/prefix_constant_rho/dataset_n_10000000_d_<d>_p_50_s_21041998.bin
```

I risultati sono salvati in:

```
results/prefix_constant_rho/streaming/.../results_streaming.csv
```

Le metriche principali osservate sono:

- cardinalità reale del prefisso $F_0(t)$ in `f0_mean_t`;
- stima media $\hat{F}_0(t)$ in `f0_hat_mean_t`;
- dispersione della stima con bande $\mu \pm \sigma$;
- errore relativo medio ai punti finali osservati, usato come sintesi comparativa tra algoritmi.

5.2.2 ANALISI DELLA ROBUSTEZZA DEGLI ALGORITMI

L'obiettivo di questa parte è capire rispetto a quali parametri gli algoritmi si comportino bene o male. I due parametri considerati sono il numero reale di elementi distinti osservati in un prefisso e il rapporto tra elementi processati e distinti, cioè il numero medio di ripetizioni del prefisso.

L'analisi tiene conto di due aspetti diversi della qualità della stima. Il primo è il bias, cioè lo scostamento sistematico tra il valore medio stimato e il numero corretto di elementi distinti. Il secondo è la robustezza, cioè la dispersione della stima attorno al suo valore medio: uno stimatore può anche essere corretto in media, ma restare comunque molto sensibile al rumore e quindi presentare una varianza elevata. Per questa ragione, nei grafici vengono mostrati insieme il valore medio della stima e la sua dispersione.

Per studiare separatamente la dipendenza da questi due parametri, sono stati adottati due protocolli sperimentali distinti.

Nel primo caso, chiamato per facilità caso A , si fissa il rapporto

$$\rho = \frac{n}{d}$$

e si osserva l'andamento della stima al variare del numero reale di elementi distinti presenti nel prefisso. In questo modo si analizza come cambino bias e dispersione a parità di ripetizione media.

Nel secondo caso, chiamato caso B , si fissano alcuni valori target della cardinalità reale e si osserva la stima in funzione del rapporto medio di ripetizione del prefisso, calcolato come

$$\rho_t = \frac{t}{F_0(t)}$$

Questa scelta è necessaria perché usare il numero grezzo di elementi processati come ascissa introdurrebbe implicitamente anche una dipendenza dal numero di elementi distinti. Il rapporto ρ_t permette invece di isolare meglio l'effetto delle ripetizioni medie sul comportamento degli algoritmi.

STIMA IN FUNZIONE DELLA CARDINALITÀ REALE DEL PREFISSO

Questa analisi corrisponde ai grafici precedentemente indicati come analisi del caso A . L'ascissa riporta la cardinalità reale $F_0(t)$, l'ordinata la stima $\hat{F}_0(t)$, e i pannelli sono separati per ciascun valore di ρ .

STIMA IN FUNZIONE DEL RAPPORTO TRA ELEMENTI PROCESSATI E DISTINTI

Questa analisi corrisponde ai grafici precedentemente indicati come analisi del caso B . Per ciascun valore obiettivo

$$F_0^* \in \{1000, 2000, 5000, 10000, 20000, 50000, 100000\}$$

si estrae per interpolazione il punto la cui ascissa è il rapporto

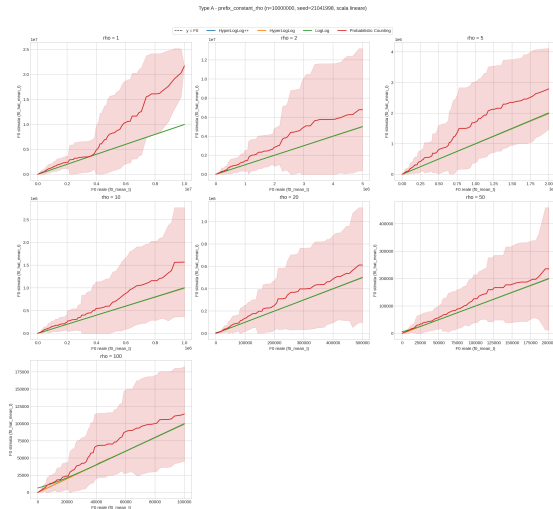
$$\rho_t = \frac{t}{F_0(t)}$$

e la cui ordinata è la stima corrispondente.

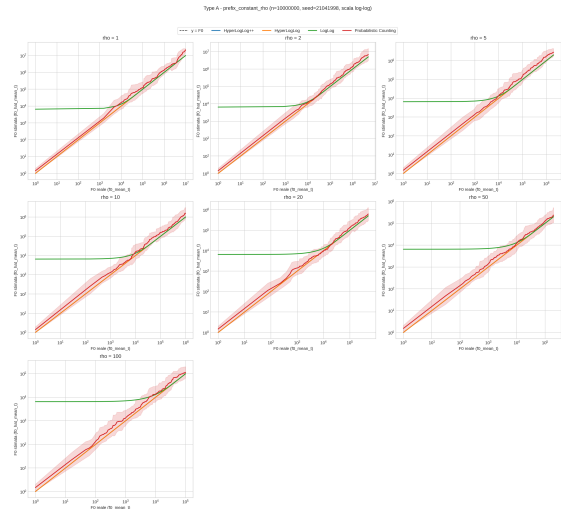
RISULTATI SULLA ROBUSTEZZA E SUL BIAS

Le osservazioni principali emerse dai riassunti ai punti finali osservati sono:

- nella vista rispetto alla cardinalità reale del prefisso, HyperLogLog e HyperLogLog++ risultano quasi sovrapposti, con errore relativo medio ai punti finali di circa 0.00618;
- nella stessa vista, LogLog segue con errore medio di circa 0.00708, mentre Probabilistic Counting è nettamente più instabile con errore medio di circa 0.770;
- nella vista rispetto al rapporto tra elementi processati e distinti, HyperLogLog++ è l'algoritmo più accurato con errore medio di circa $3.95 \cdot 10^{-4}$;

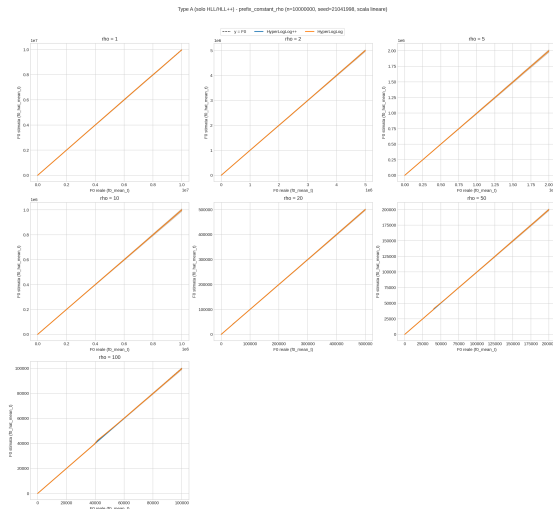


(a) Scala lineare

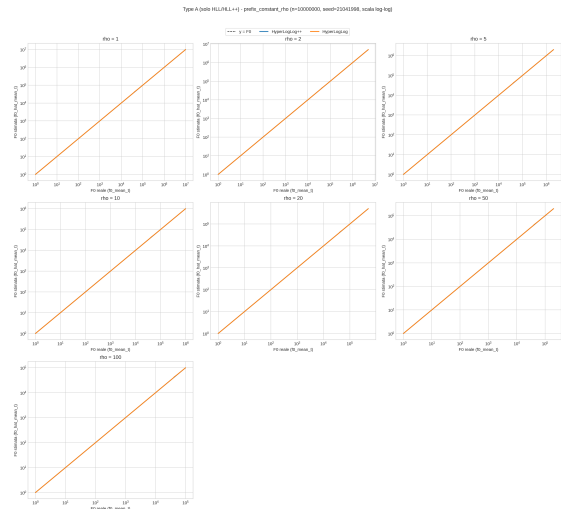


(b) Scala logaritmica

Figura 5.3: Stima rispetto alla cardinalità reale del prefisso, tutti gli algoritmi



(a) Scala lineare



(b) Scala logaritmica

Figura 5.4: Stima rispetto alla cardinalità reale del prefisso, confronto HyperLogLog e HyperLogLog++

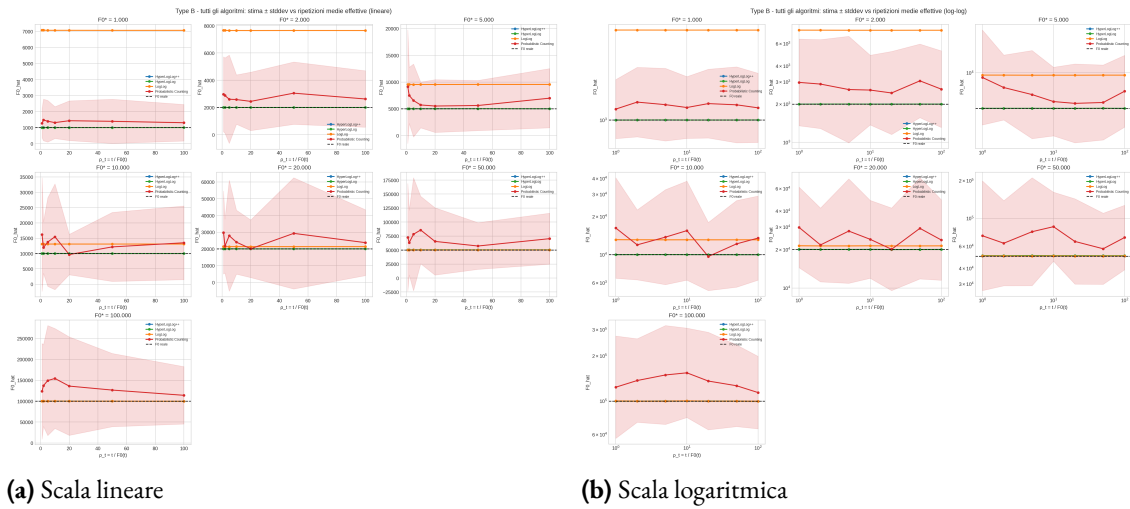


Figura 5.5: Stima rispetto al rapporto tra elementi processati e distinti, tutti gli algoritmi

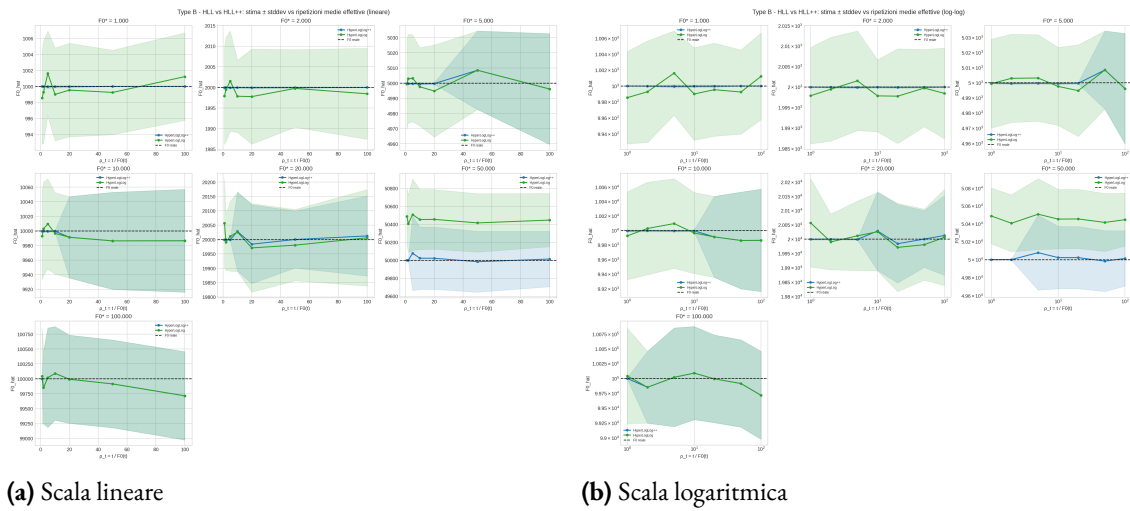


Figura 5.6: Stima rispetto al rapporto tra elementi processati e distinti, confronto HyperLogLog e HyperLogLog++

- HyperLogLog segue con errore medio di circa $2.08 \cdot 10^{-3}$;
- nella stessa vista, Probabilistic Counting ha errore medio di circa 0.346;
- nella seconda vista, l'errore medio più alto è osservato per LogLog, con valore di circa 1.455.

Le Figure 5.4 e 5.6 confermano che HyperLogLog++ mantiene il comportamento più stabile nelle due viste complementari del problema.

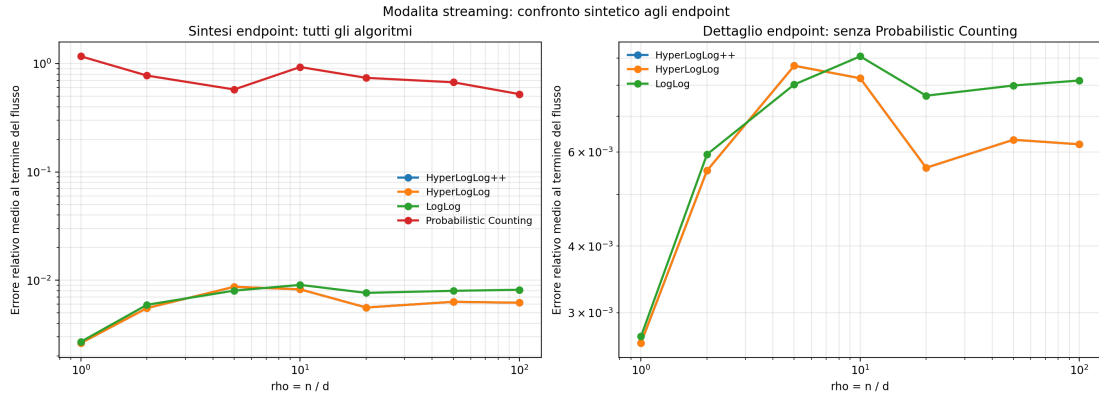


Figura 5.7: Errore relativo medio alla fine della stream. Il pannello sinistro confronta tutti gli algoritmi, mentre quello destro isola HyperLogLog++, HyperLogLog e LogLog

La Figura 5.7 riassume in forma compatta la stessa evidenza: Probabilistic Counting resta nettamente separato dagli altri algoritmi, mentre HyperLogLog++ mantiene il comportamento più regolare e accurato nel confronto complessivo. Questo risultato è coerente con la letteratura su HLL++, in cui la riduzione empirica del bias e la gestione separata dei regimi di cardinalità piccola e intermedia spiegano il vantaggio rispetto a HLL classico [7]. Più in generale, i grafici osservati restano compatibili con la linea evolutiva che porta da LogLog a HyperLogLog e poi a HyperLogLog++, cioè a una progressiva riduzione della varianza e del bias pratico [5, 6, 7].

5.2.3 ANALISI SULLA VARIAZIONE DEL PARAMETRO DEGLI ALGORITMI

Per analizzare il comportamento degli algoritmi al variare della precisione, è necessario isolare l'effetto del parametro.

Di conseguenza, è stato fatto un secondo insieme di esperimenti in cui sono stati fissati lo stesso insieme di dati, lo stesso seed e la stessa funzione di hash, variando solo il parametro interno di ciascun algoritmo.

L'impostazione scelta per questo esperimento comprende:

- l'utilizzo della funzione di hash `splitmix64`
- un seed fissato a 21041998
- tre valori rappresentativi per $\rho \in \{1, 10, 100\}$
- HyperLogLog++ con $k \in \{8, 10, 12, 14, 16, 18\}$
- HyperLogLog e LogLog con $k \in \{4, 6, 8, 10, 12, 14, 16\}$ e $L = 32$

- Probabilistic Counting con $L \in \{4, 8, 12, 16, 20, 24, 28, 31\}$

Per analizzare il comportamento degli algoritmi al variare della precisione, sono stati realizzati due tipi di grafici.

Il primo, tramite una heatmap, riassume l'errore relativo medio alla fine del flusso di streaming per ogni coppia di valori tra ρ e il parametro dell'algoritmo.

Il secondo sia in scala lineare che in scala logaritmica, fissando ρ , mette a confronto la cardinalità reale del flusso e la stima, dove ogni retta è una diversa precisione dell'algoritmo.

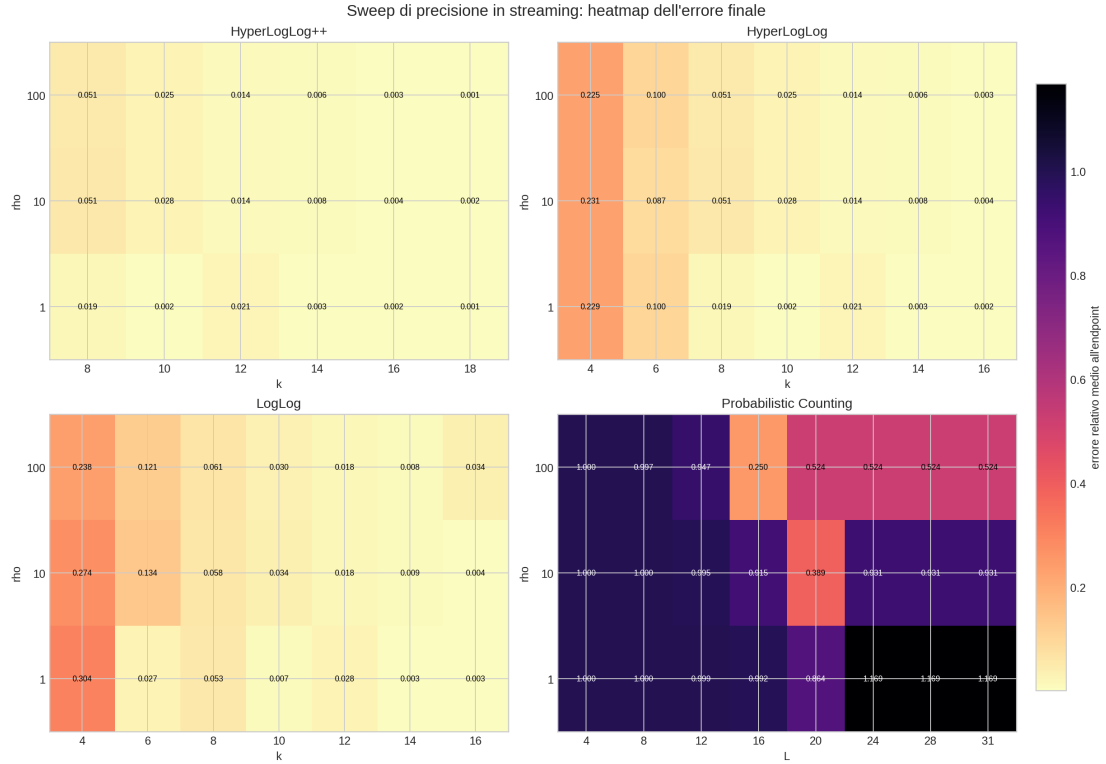


Figura 5.8: Errore relativo medio alla fine della stream al variare del parametro di precisione e di ρ , per ciascun algoritmo

Algoritmo	Complessivo	$\rho = 1$	$\rho = 10$	$\rho = 100$
HyperLogLog++	$k = 18$	$k = 18$	$k = 18$	$k = 18$
HyperLogLog	$k = 16$	$k = 10$	$k = 16$	$k = 16$
LogLog	$k = 14$	$k = 14$	$k = 16$	$k = 14$
Probabilistic Counting	$L = 20$	$L = 20$	$L = 20$	$L = 16$

Tabella 5.1: Miglior parametro osservato per algoritmo nell'analisi del parametro di precisione

RISULTATI SULLA PRECISIONE PER OGNI ALGORITMO

Le Figure 5.8 e 5.9 mostrano che l'aumento del parametro di precisione non produce lo stesso effetto per tutti gli algoritmi.

HyperLogLog++ è l'algoritmo con l'andamento più regolare. Nel dominio osservato, all'aumentare di k l'errore tende a ridursi in modo stabile e il valore migliore coincide sempre con $k = 18$, indipendentemente dal valore di ρ . Questo comportamento indica che l'algoritmo non solo è accurato in media, ma risponde anche in modo prevedibile all'aumento delle risorse disponibili.

HyperLogLog mostra una tendenza simile, ma meno uniforme. In media il parametro migliore è $k = 16$, tuttavia nel caso $\rho = 1$ il beneficio si stabilizza già attorno a $k = 10$. L'aumento del parametro continua quindi a migliorare la stima, ma con un guadagno meno regolare rispetto a HyperLogLog++.

LogLog presenta invece un comportamento più irregolare. Il parametro migliore medio risulta $k = 14$, ma il vantaggio non cresce in modo monotono al crescere di k : il valore $k = 16$ migliora il caso intermedio $\rho = 10$, ma non si mantiene ottimale negli altri regimi osservati. Questo suggerisce una maggiore sensibilità alla scelta del parametro.

Probabilistic Counting resta nettamente separato dagli altri algoritmi. Anche nel caso del parametro migliore, pari a $L = 20$, l'errore finale rimane molto più elevato e il miglioramento ottenuto aumentando L non è sufficiente a portare l'algoritmo su livelli comparabili con quelli delle famiglie a registri.

Questo esperimento aggiunge quindi un'informazione che i confronti precedenti non mostravano direttamente. Non basta osservare quale algoritmo sia più accurato nel complesso: è altrettanto importante capire se la qualità della stima migliori in modo regolare al variare del parametro. Sotto questo aspetto, HyperLogLog++ risulta l'algoritmo più stabile e più robusto, mentre LogLog e soprattutto Probabilistic Counting mostrano una dipendenza molto più marcata dalla scelta del parametro. Anche questa evidenza è in linea con la dipendenza classica dell'errore dal numero di registri, che nelle famiglie LogLog e HyperLogLog segue un andamento dell'ordine di $1/\sqrt{m}$ [5, 6].

5.3 ESPERIMENTI SUL MERGE DISTRIBUITO

Tutti gli esperimenti sul merge sono stati fatti sull'algoritmo HyperLogLog++. Essi sono poi suddivisi secondo tre casi distinti:

- `valid`: merge semanticamente corretto in modo diretto;
- `invalid`: merge non semanticamente corretto;
- `recoverable`: merge non direttamente corretto, ma recuperabile con normalizzazione esplicita.

La possibilità di costruire sketch locali e combinarli senza rivedere i dati originari è uno dei motivi principali per cui queste strutture vengono usate in contesti distribuiti e in sistemi che aggregano grandi collezioni di dataset [16, 29].

La sperimentazione del merge omogeneo implementa il caso `valid`, mentre i due esperimenti eterogenei permettono di osservare separatamente un caso `invalid` e un caso `recoverable`.

Nella lettura dei risultati eterogenei è utile osservare anche gli scenari invertiti tra lato sinistro e lato destro, indicati con il suffisso `rev`. Questa scelta non serve a verificare una nuova proprietà algebrica del merge eterogeneo,

ma a controllare se il degrado osservato cambi quando si scambiano i due contesti locali. La proprietà matematica di commutatività del merge è invece già verificata nei test del caso omogeneo, insieme all'idempotenza e alla coerenza con il riferimento seriale.

5.3.1 CASO OMOGENEO

Affinché si possa fare degli esperimenti nel caso di merge omogeneo, è necessario effettuare il merge mantenendo identici la stessa funzione di hash, lo stesso seed e gli stessi parametri dell'algoritmo.

La configurazione minima scelta prevede:

$$\rho \in \{1, 10, 100\}, \quad k \in \{10, 14, 18\}$$

sono state analizzate 36 configurazioni e 900 coppie, combinando i tre valori di k , i tre regimi di ρ e le quattro funzioni di hash disponibili.

In tutte le 900 coppie analizzate si osserva:

$$\Delta_{abs} = 0, \quad \Delta_{rel} = 0$$

La tabella seguente riassume lo stesso risultato nei tre regimi di ρ considerati.

ρ	d	Configurazioni	Coppie totali	Coppie con delta $\neq 0$	Max Δ_{abs}	Max Δ_{rel}
1	10^7	12	300	0	0	0
10	10^6	12	300	0	0	0
100	10^5	12	300	0	0	0

Tabella 5.2: Risultati del controllo omogeneo di HyperLogLog++

Il controllo omogeneo costituisce il riferimento del caso `valid` e mostra che, nel dominio osservato, merge e l'unione seriale coincidono in modo sistematico. Questo esperimento fornisce quindi la base rispetto alla quale leggere gli scostamenti osservati nei casi eterogenei. Il risultato è coerente con la semantica classica del merge per gli sketch HLL-like, che in condizioni omogenee coincide con lo stato ottenuto processando direttamente l'unione dei flussi [21].

5.3.2 CASO ETEROGENEO

Nel caso di merge eterogeneo è stato deciso di analizzare due precisi casi distinti. Lo scopo di questi esperimenti era cercare di analizzare quanto potesse degradare il merge in condizioni non ottimali, come ad esempio in un contesto di nodi distribuiti, in cui ogni nodo aveva a disposizione poca memoria e che fosse necessario utilizzare precisioni o parametri differenti.

Il primo esperimento consiste nell'analisi sull'utilizzo di funzioni di hash differenti, che rappresenta un caso `invalid`, mentre il merge in caso di precisioni differenti appartiene ai casi `recoverable` e richiede quindi una lettura diversa dei risultati.

HASH MISMATCH

La prima analisi eterogenea riguarda il caso in cui i due sketch vengono costruiti usando funzioni di hash differenti, oppure la stessa famiglia di hash ma con seed diversi. L'obiettivo è capire come si comporti l'unione di sketch quando la semantica dell'hash non è più condivisa tra i due lati.

Da questo esperimento ci si aspetta sicuramente una degradazione della stima. Poiché lo stesso numero rappresentato da due funzioni di hash differenti verrà rappresentato diversamente. L'esperimento vuole vedere quanto effettivamente viene degradata la stima.

La configurazione considerata è stata:

$$n = 10^7, \quad p = 50, \quad seed = 21041998, \quad d \in \{10^5, 10^6, 10^7\}$$

da cui si ricava

$$\rho \in \{100, 10, 1\}$$

con precisione fissata a $k = 14$.

Le funzioni di hash considerate in questa configurazione minima sono `splitmix64`, `xxhash64`, `murmurhash3` e `siphash24`. Gli scenari osservati sono sette:

- `ctrl`: controllo omogeneo locale, con la stessa funzione di hash sui due lati e seed del dataset su entrambi;
- `seed_xx` e `seed_xx_rev`: stessa hash `xxhash64`, ma seed diversi nei due lati;
- `seed_murmur` e `seed_murmur_rev`: stessa hash `murmurhash3`, ma seed diversi nei due lati;
- `family` e `family_rev`: confronto tra `xxhash64` e una funzione di hash diversa tra `splitmix64`, `murmurhash3` e `siphash24`.

I casi di *seed mismatch* sono stati testati solo su `xxhash64` e `murmurhash3`, perché in questa configurazione minima sono le due famiglie per cui l'utilizzo di seed differenti è stato usato come fattore di eterogeneità. Nei casi `family` e `family_rev`, invece, l'eterogeneità riguarda direttamente l'utilizzo di funzioni di hash.

Lo scenario `ctrl` è presente anche in questo esperimento come riferimento locale nella stessa figura e nella stessa tabella. Non rappresenta un esperimento distinto dal caso omogeneo discusso in precedenza, ma un suo sottoinsieme su `HyperLogLog++` con $k = 14$, utile per confrontare direttamente i casi `invalid` con il comportamento corretto.

Nel controllo omogeneo la strategia usata è `direct`. Nei casi `invalid`, invece, vengono osservate due strategie: `reject` e `unsafe_naive_merge`. La prima non produce una stima di merge e registra esplicitamente il caso incompatibile. La seconda forza il merge come se la coppia fosse compatibile, ed è quindi usata come controllo negativo sperimentale.

Questo controllo serve quindi a distinguere i casi in cui il merge è trattato come semanticamente valido dai casi in cui viene forzato esplicitamente come controllo negativo.

L'esperimento copre 1800 righe complessive, distribuite su 72 file in formato CSV:

- 300 coppie nel controllo omogeneo;
- 750 coppie `invalid` con `reject`;

- 750 coppie invalid con unsafe_naive_merge.

La Figura 5.10 mostra, per i diversi valori di ρ , il contrasto tra il merge omogeneo locale e il merge forzato nei casi invalid. La Tabella 5.3 ne riassume i principali valori numerici. Le quantità $\bar{\varepsilon}_{merge}$ e $\bar{\varepsilon}_{serial}$ indicano rispettivamente l'errore relativo medio del merge e del riferimento seriale. La colonna finale riporta invece il degrado medio, cioè la media del rapporto punto per punto tra errore del merge ed errore del riferimento seriale.

Scenario	Coppie	$\bar{\varepsilon}_{merge}$	$\bar{\varepsilon}_{serial}$	$\max \varepsilon_{merge}$	Degrado medio
ctrl	300	0.006233	0.006233	0.025913	1.00
seed_xx	75	0.342374	0.007436	0.969548	49.45
seed_xx_rev	75	0.343830	0.010076	0.969548	28.82
seed_murmur	75	0.347522	0.004836	0.984223	206.46
seed_murmur_rev	75	0.348319	0.004606	0.984223	307.18
family	225	0.352037	0.007436	1.009634	50.36
family_rev	225	0.352802	0.005832	1.009634	130.76

Tabella 5.3: Risultati del merge eterogeneo con funzioni di hash differenti

Dai risultati emerge che, nei casi invalid, il merge forzato produce un degrado netto rispetto al riferimento seriale e conferma che l'eterogeneità dell'hash rompe la semantica comune necessaria al merge. Nel regime più critico, $\rho = 1$, l'errore relativo medio raggiunge valori prossimi o superiori a 1, con massimo pari a 1.009634. La strategia reject, al contrario, evita di trattare queste coppie come merge validi e mantiene nel protocollo sperimentale soltanto la stima seriale e l'unione esatta.

PRECISION MISMATCH

La seconda analisi eterogenea riguarda l'esperimento in cui la funzione di hash resta costante, ma i due sketch vengono costruiti con precisioni differenti, cioè con parametri differenti per l'algoritmo, in questo caso cambia la quantità di registri utilizzati.

L'obiettivo è capire se l'utilizzo di parametri differenti rende il merge direttamente invalido oppure se il caso possa essere recuperato tramite una normalizzazione esplicita.

La configurazione considerata è stata:

$$n = 10^7, \quad p = 50, \quad seed = 21041998, \quad d \in \{10^5, 10^6, 10^7\}$$

da cui si ricava

$$\rho \in \{100, 10, 1\}$$

con funzione di hash fissata a xxhash64 e seed di hash identico sui due lati, pari a 21041998.

Le coppie di precisione osservate sono:

$$(k_{left}, k_{right}) \in \{(10, 14), (14, 10), (10, 18), (18, 10), (14, 18), (18, 14)\}$$

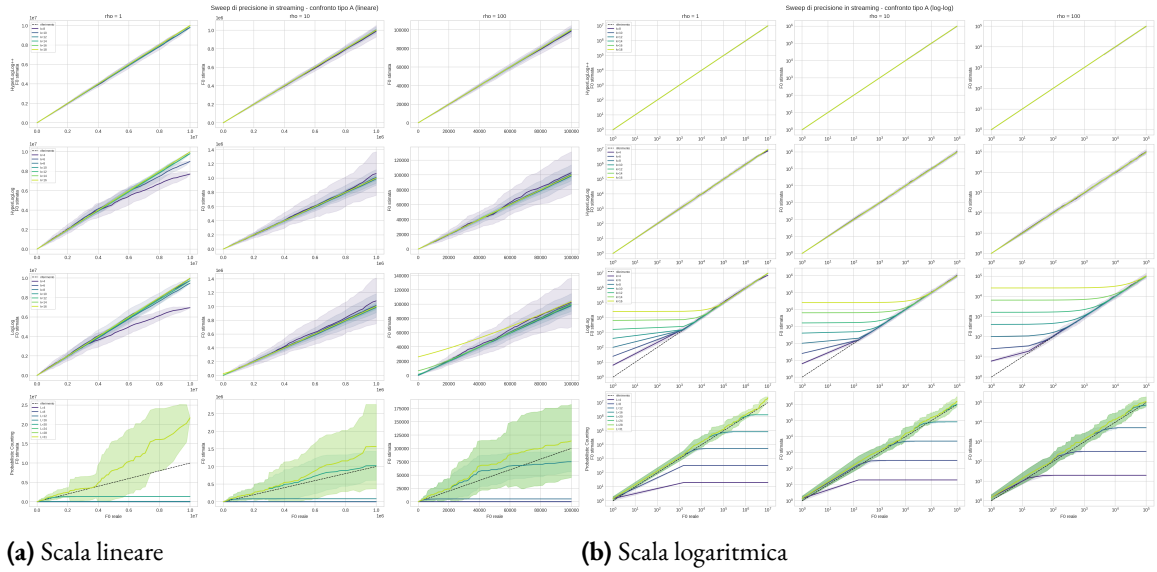


Figura 5.9: Confronto rispetto alla cardinalità reale del prefisso nell'analisi del parametro di precisione. Le righe corrispondono agli algoritmi e le colonne ai tre valori di ρ

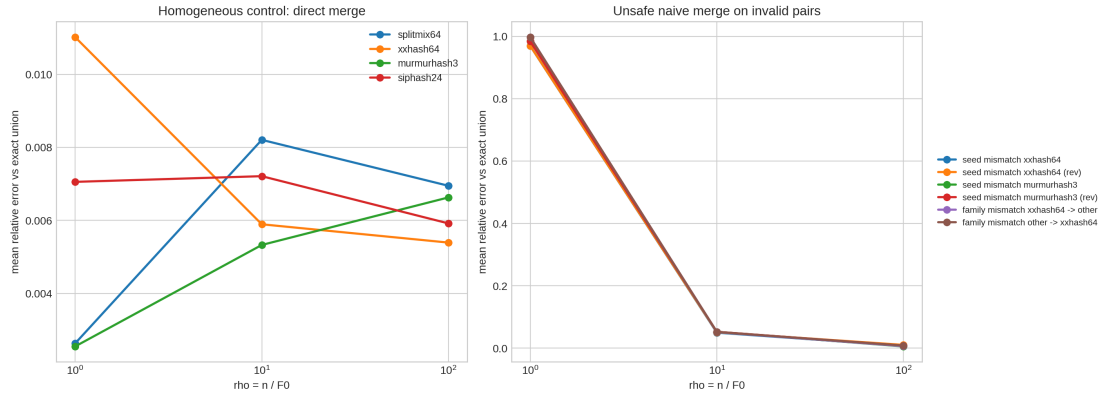


Figura 5.10: Esperimento di merge eterogeneo con funzioni di hash differenti

Questi sei casi vengono letti in tre gruppi di precisione: 10 vs 14, 10 vs 18 e 14 vs 18, ciascuno osservato nelle due direzioni tra lato sinistro e lato destro.

In tutti questi casi la coppia è classificata come *recoverable*. Per questa ragione vengono osservate tre strategie: *reject*, *unsafe_naive_merge* e *reduce_then_merge*. La prima non produce una stima di merge e registra esplicitamente il caso. La seconda forza il merge senza alcuna normalizzazione preventiva. La terza riduce entrambi gli sketch alla precisione più bassa compatibile e applica poi il merge nello spazio comune.

Questo controllo serve quindi a distinguere i casi in cui il parametro di precisione è differente e che venga trattato come merge recuperabile dai casi in cui il merge viene forzato senza normalizzazione.

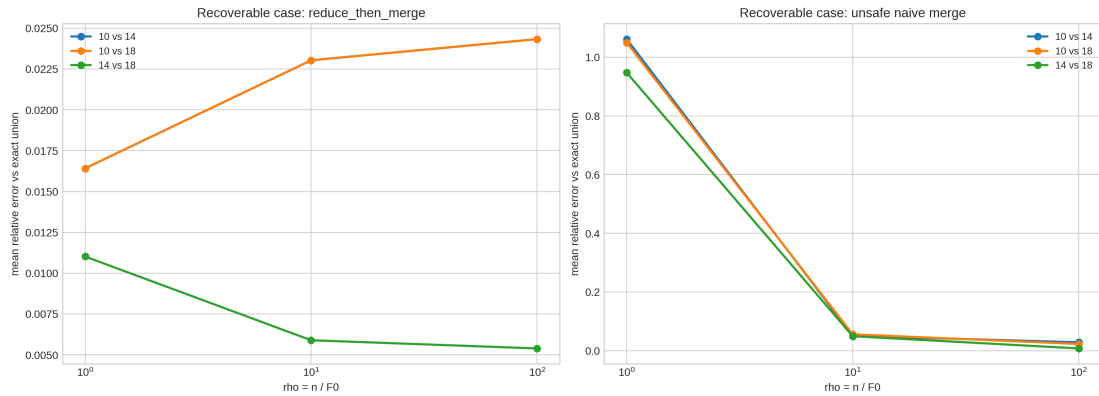
L'esperimento copre 1350 righe complessive, distribuite su 54 file in formato CSV:

- 450 coppie *recoverable* con *reject*;
- 450 coppie *recoverable* con *unsafe_naive_merge*;
- 450 coppie *recoverable* con *reduce_then_merge*.

La Tabella 5.4 riassume i principali valori numerici dell'esperimento. Le quantità $\bar{\varepsilon}_{merge}$ e $\bar{\varepsilon}_{serial}$ indicano rispettivamente l'errore relativo medio del merge e del caso seriale. Anche in questo caso l'ultima colonna riporta il degrado medio, cioè la media del rapporto punto per punto tra errore del merge ed errore del merge seriale. La riga *reject* non corrisponde a una stima di merge, ma a una strategia di rifiuto; per questo le colonne relative al merge restano non valorizzate.

Strategia	Coppie	$\bar{\varepsilon}_{merge}$	$\bar{\varepsilon}_{serial}$	$\max \varepsilon_{merge}$	Degrado medio
<i>reject</i>	450	–	0.016651	–	–
<i>unsafe_naive_merge</i>	450	0.363882	0.016651	1.061378	32.37
<i>reduce_then_merge</i>	450	0.016651	0.016651	0.057418	1.00

Tabella 5.4: Risultati del merge eterogeneo nel caso in cui i parametri degli algoritmi non coincidono



(a) Confronto tra *reduce_then_merge* e *unsafe_naive_merge* nel caso in cui i parametri degli algoritmi non coincidono

CORRETTEZZA DELLA NORMALIZZAZIONE `reduce_then_merge` Dai risultati emerge che, nei casi `recoverable`, la normalizzazione alla precisione minima elimina lo scostamento introdotto dall'utilizzo di parametri differenti. Con l'utilizzo della strategia `reduce_then_merge`, il comportamento coincide infatti sia con il riferimento seriale sia con il riferimento omogeneo alla precisione più bassa, con degrado medio pari a 1.00. Questo comportamento è coerente con la letteratura più recente sugli sketch HyperLogLog, che tratta la riduzione alla precisione minima come passaggio corretto per rendere confrontabili sketch costruiti con parametri diversi [21].

DEGRADO DEL MERGE FORZATO NEL CASO `recoverable` Quando la normalizzazione non viene applicata, il merge forzato mostra invece un degrado netto rispetto al riferimento seriale. Nel gruppo 10 vs 14, l'errore relativo medio passa da circa 0.0290 per $\rho = 100$ a 0.0509 per $\rho = 10$, fino a raggiungere 1.061378 per $\rho = 1$. Nel gruppo 10 vs 18 i valori sono molto simili, con massimo 1.049045 nel regime $\rho = 1$. Nel gruppo 14 vs 18 il degrado resta più contenuto per $\rho = 100$, ma cresce nuovamente fino a 0.947067 quando $\rho = 1$. Anche in questo caso, quindi, il merge forzato non si limita a introdurre una perdita moderata di accuratezza, mentre la strategia `reduce_then_merge` riporta il comportamento al riferimento corretto.

5.4 RISULTATI OTTENUTI

I risultati presentati in questo capitolo possono essere riassunti in quattro punti principali.

- **Algoritmo migliore: HyperLogLog++.** Nel dominio sperimentale osservato, HyperLogLog++ risulta il metodo migliore perché combina *accuratezza media più alta, minore dispersione della stima e comportamento più regolare al variare del parametro di precisione*. Le Figure 5.4, 5.6 e 5.7 mostrano infatti che HyperLogLog++ mantiene la stima più stabile sia rispetto alla cardinalità reale del prefisso sia rispetto al rapporto tra elementi processati e distinti. La Tabella 5.1 conferma inoltre che, durante l'esperimento sui possibili parametri, HyperLogLog++ è anche l'algoritmo con la risposta più prevedibile all'aumentare di k , con valore migliore osservato sempre pari a $k = 18$.
- **Coincidenza tra merge omogeneo e riferimento seriale.** Quando due sketch condividono la stessa funzione di hash, lo stesso seed e gli stessi parametri, il merge coincide con il caso seriale nel dominio osservato. La Tabella 5.2 riporta infatti $\Delta_{abs} = 0$ e $\Delta_{rel} = 0$ in tutte le coppie analizzate. Questo è il risultato di riferimento del caso `valid` e mostra che, in condizioni omogenee, il merge preserva correttamente la semantica dello sketch.
- **Hash diverse nel merge compromettono la correttezza.** Quando i due sketch sono costruiti con funzioni di hash diverse, oppure con seed diversi della stessa famiglia di hash, il merge produce un degrado netto rispetto al riferimento seriale. La Tabella 5.3 mostra che l'errore relativo medio del merge cresce di oltre un ordine di grandezza e, nel regime più critico $\rho = 1$, può superare il valore 1. Questo conferma che il merge richiede non solo lo stesso algoritmo, ma anche una semantica comune dell'hash.
- **La tecnica `reduce_then_merge` recupera il caso di precisioni diverse.** Se i due sketch usano la *stessa funzione di hash* ma *precisioni diverse*, il merge non è direttamente corretto, ma può essere recuperato normalizzando entrambi gli sketch alla precisione minima comune prima dell'unione. La Tabella 5.4 mostra

che, con `reduce_then_merge`, l'errore medio del merge coincide con quello del riferimento seriale e del corrispondente caso omogeneo, mentre il merge senza normalizzazione introduce un degrado netto.

Complessivamente, il risultato principale del capitolo è che HyperLogLog++ emerge come il miglior stimatore tra quelli implementati, mentre sul lato del merge la correttezza dipende in modo decisivo dalla compatibilità semantica tra gli sketch: il caso omogeneo coincide con il seriale, l'eterogeneità dell'hash rompe la correttezza, e quando due sketch hanno precisioni diverse il merge resta recuperabile solo attraverso una normalizzazione esplicita.

6

Conclusioni

Il conteggio approssimato degli elementi distinti è un problema centrale nei flussi di dati, perché compare in contesti come telemetria, osservabilità, monitoraggio e analisi distribuita, dove il costo del conteggio esatto cresce rapidamente al crescere dei dati osservati. In questi contesti gli sketch permettono di mantenere stime compatte e aggiornabili in tempo ridotto, ma il punto critico non è soltanto la qualità della stima locale. Nei sistemi distribuiti conta altrettanto la possibilità di effettuare il merge di sketch prodotti su nodi diversi, mantenendo una semantica coerente.

Il lavoro sviluppato ha affrontato questo problema su due livelli. Il primo è il confronto tra algoritmi per il calcolo della stima degli elementi distinti in una stream, con una stessa infrastruttura di esecuzione, una stessa interfaccia comune e un dataset sintetico con proprietà controllate. Il secondo è lo studio del merge distribuito, prima nel caso omogeneo e poi in due casi eterogenei mirati, cioè come la funzione di hash e il cambio dei parametri all'interno degli algoritmi cambino la stima. Questa doppia prospettiva ha permesso di mettere in relazione il comportamento della stima con il problema, più delicato, della compatibilità semantica tra sketch.

I risultati sperimentali mostrano che, nel dominio osservato, HyperLogLog++ mantiene il comportamento più regolare in una stream di dati, sia quando la stima è letta rispetto alla cardinalità reale del prefisso, sia quando viene analizzata rispetto al rapporto tra elementi processati e distinti. L'analisi del parametro di precisione conferma la stessa tendenza: l'aumento di k produce in HyperLogLog++ un miglioramento più stabile e prevedibile di quanto si osservi negli altri algoritmi implementati. HyperLogLog resta il metodo più vicino per comportamento, LogLog mostra maggiore sensibilità alla scelta del parametro e Probabilistic Counting rimane nettamente più instabile nel dominio testato.

Il controllo sul merge omogeneo conferma che, quando i due lati condividono la stessa funzione di hash, lo stesso seed e gli stessi parametri, il merge coincide con il riferimento seriale nel dominio osservato. Questa verifica era necessaria per distinguere gli effetti dovuti all'eterogeneità da eventuali problemi dell'infrastruttura sperimentale. A partire da questo controllo, gli esperimenti eterogenei hanno mostrato due comportamenti distinti. Nel caso di hash mismatch, il merge forzato produce un degrado netto e conferma che l'eterogeneità dell'hash rompe

la semantica comune necessaria al merge. Nel caso di `precision mismatch`, invece, la differenza nei parametri non rende il caso direttamente corretto, ma resta recuperabile tramite una normalizzazione esplicita alla precisione minima, che riporta il comportamento sul riferimento seriale e sul corrispondente riferimento omogeneo.

Da questi risultati emergono anche i contributi principali del lavoro. Il primo è l'implementazione uniforme di più algoritmi di sketching per il calcolo degli elementi distinti, con particolare attenzione alla linea evolutiva che arriva a HyperLogLog++. Il secondo è l'integrazione delle funzioni di hash sotto una stessa interfaccia, in modo da poterle confrontare a parità di contesto sperimentale. Il terzo è l'infrastruttura di valutazione, che separa dataset, algoritmi, hash, metriche e serializzazione dei risultati e permette di condurre esperimenti sia in modalità di flusso sia nel merge distribuito. Il quarto è la distinzione operativa tra casi `valid`, `recoverable` e `invalid`, che rende osservabile in modo esplicito quando il merge sia semanticamente corretto, quando vada rifiutato e quando possa essere recuperato con una trasformazione definita.

Il sistema sviluppato ha però anche delle limitazioni. Gli esperimenti sono basati soprattutto su un dataset sintetico controllato, utile per isolare i fenomeni che interessano ma non sufficiente da solo a rappresentare tutta la varietà dei flussi reali. Lo studio del merge è stato condotto principalmente su casi *pairwise*, che bastano a chiarire il problema semantico di base ma non esauriscono la complessità delle topologie distribuite più ampie. Inoltre i casi eterogenei analizzati si concentrano su due assi ben definiti, cioè `hash mismatch` e `precision mismatch`, e la parte sperimentale sul merge è stata sviluppata soprattutto su HyperLogLog++ come riferimento principale. Infine il framework non è ancora collegato direttamente a una sorgente reale di flusso continuo.

6.1 SVILUPPI FUTURI

Le direzioni più naturali che emergono da questo lavoro sono diverse. Una prima direzione consiste nel portare il framework fuori dal solo contesto sintetico, usando dataset reali oppure integrando una sorgente di flusso come Apache Kafka, così da osservare il comportamento degli algoritmi e del merge anche su dati prodotti da sistemi complessi reali.

Una seconda direzione riguarda le topologie distribuite del merge. I risultati ottenuti sul `precision mismatch` indicano che, quando la differenza di precisione è recuperabile, il merge tende a ridursi alla precisione minima coinvolta nella catena. Per questo motivo non avrebbe senso studiare topologie arbitrarie senza tenere conto di questo vincolo. Ha invece senso analizzare in quali punti di una topologia distribuita convenga effettuare la riduzione alla precisione più bassa, in modo da minimizzare il numero di conversioni e controllare il costo complessivo del merge.

Una terza direzione consiste nell'estendere l'analisi eterogenea ad assi diversi da quelli già studiati, mantenendo però lo stesso principio: osservare solo casi la cui interpretazione semantica sia chiara. Questo include, per esempio, ulteriori configurazioni di hash o di rappresentazione interna da valutare con lo stesso metodo usato nei casi `invalid` e `recoverable` già discussi.

Una quarta direzione riguarda l'estensione della famiglia di algoritmi implementati. In particolare avrebbe senso integrare algoritmi più recenti della linea LogLog, come UltraLogLog e metodi affini, per verificare se lo stesso impianto sperimentale permetta di confrontarli in modo diretto con HyperLogLog++ sia nella stima in flusso sia nel merge.

Una quinta direzione consiste nell'allargare il framework anche ad altre famiglie di sketch. Bloom filter e Count-Min Sketch mostrano già nel capitolo sullo stato dell'arte che il paradigma dello sketching non si esaurisce nel solo numero di distinti. Integrare nel framework algoritmi per appartenenza approssimata, stima di frequenze o altri problemi permetterebbe di verificare quanto le stesse idee di compatibilità e di merge possano essere riutilizzate anche oltre il caso di HyperLogLog.

Infine, una direzione ulteriore riguarda la misura del costo del merge eterogeneo non solo in termini di errore finale, ma anche in termini di costo di normalizzazione, costo computazionale e costo cumulativo lungo una topologia di elaborazione. Questo renderebbe il quadro non soltanto più completo dal punto di vista sperimentale, ma anche più utile dal punto di vista ingegneristico.

Bibliografia

- [1] J. Leskovec, A. Rajaraman, and J. D. Ullman, *Mining Data Streams*. Cambridge University Press, 2014, p. 123–153.
- [2] S. Muthukrishnan, “Data streams: Algorithms and applications,” Rutgers University, Tech. Rep., 2005.
- [3] G. Cormode, “Data sketching,” *Communications of the ACM*, 2017.
- [4] P. Flajolet and G. N. Martin, “Probabilistic counting algorithms for data base applications,” *Journal of Computer and System Sciences*, vol. 31, no. 2, 1985.
- [5] M. Durand and P. Flajolet, “Loglog counting of large cardinalities,” in *Proceedings of the European Symposium on Algorithms (ESA 2003)*, 2003.
- [6] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier, “Hyperloglog: the analysis of a near-optimal cardinality estimation algorithm,” in *Proceedings of the 2007 Conference on Analysis of Algorithms (AofA 07)*, 2007, pp. 127–146.
- [7] S. Heule, M. Nunkesser, and A. Hall, “Hyperloglog in practice: Algorithmic engineering of a state of the art cardinality estimation algorithm,” in *Proceedings of the International Conference on Extending Database Technology (EDBT/ICDT ’13)*. ACM, 2013.
- [8] N. Prezza, “Algorithms for massive data – lecture notes,” 2025, lecture notes, Ca’ Foscari University of Venice.
- [9] Z. Bar-Yossef, T. S. Jayram, R. Kumar, D. Sivakumar, and L. Trevisan, “Counting distinct elements in a data stream,” in *Randomization and Approximation Techniques in Computer Science (RANDOM 2002)*, ser. Lecture Notes in Computer Science, vol. 2483. Springer, 2002, pp. 1–10.
- [10] N. Alon, Y. Matias, and M. Szegedy, “The space complexity of approximating the frequency moments,” *Journal of Computer and System Sciences*, vol. 58, no. 1, pp. 137–147, 2002.
- [11] D. M. Kane, J. Nelson, and D. P. Woodruff, “An optimal algorithm for the distinct elements problem,” in *Proceedings of the 29th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS 2010)*. ACM, 2010.
- [12] R. Motwani and P. Raghavan, “Randomized algorithms,” *ACM Computing Surveys*, vol. 28, no. 1, March 1996, copyright 1996, CRC Press.
- [13] S. Vadhan and M. Mitzenmacher, “Why simple hash functions work: Exploiting the entropy in a data stream,” 2007, manuscript.
- [14] A. Pagh and R. Pagh, “Uniform hashing in constant time and optimal space,” 2007, manuscript.

- [15] J. L. Carter and M. N. Wegman, “Universal classes of hash functions,” *Journal of Computer and System Sciences*, vol. 18, pp. 143–154, 1979.
- [16] P. K. Agarwal, G. Cormode, Z. Huang, J. M. Phillips, Z. Wei, and K. Yi, “Mergeable summaries,” in *Proceedings of the 31st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS 2012)*. ACM, 2012.
- [17] G. Cormode, “Count-min sketch,” 2010, encyclopedia entry; local source: thesis/figures/cmencyc.pdf.
- [18] B. H. Bloom, “Space/time trade-offs in hash coding with allowable errors,” *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970.
- [19] S. Agarwal and A. Trachtenberg, “Approximating the number of differences between remote sets,” 2006, technical report/manuscript; local source: thesis/figures/bloom.pdf.
- [20] S. van de Geer, “Mathematical statistics,” 2015, lecture notes, September 2015.
- [21] O. Ertl, “New cardinality estimation algorithms for hyperloglog sketches,” 2017, arXiv preprint. [Online]. Available: <https://oertl.github.io/hyperloglog-sketch-estimation-paper/paper/paper.pdf>
- [22] K. J. Lang, “Back to the future: an even more nearly optimal cardinality estimation algorithm,” 2017, arXiv preprint. [Online]. Available: <https://arxiv.org/abs/1708.06839>
- [23] O. Ertl, “Setsketch: Filling the gap between minhash and hyperloglog,” *Proceedings of the VLDB Endowment*, vol. 14, no. 11, pp. 2244–2257, 2021. [Online]. Available: <https://www.vldb.org/pvldb/vol14/p2244-ertl.pdf>
- [24] M. Karppa and R. Pagh, “Hyperlogloglog: Cardinality estimation with one log more,” 2022, arXiv preprint. [Online]. Available: <https://arxiv.org/abs/2205.11327>
- [25] O. Ertl, “Ultraloglog: A practical and more space-efficient alternative to hyperloglog for approximate distinct counting,” 2024, VLDB 2024 / arXiv preprint. [Online]. Available: <https://arxiv.org/abs/2308.16862>
- [26] P. B. Gibbons and S. Tirthapura, “Estimating simple functions on the union of data streams,” in *Proceedings of the Thirteenth Annual ACM Symposium on Parallel Algorithms and Architectures (SPAA 2001)*, 2001, pp. 281–291.
- [27] K. S. Beyer, P. J. Haas, B. Reinwald, Y. Sismanis, and R. Gemulla, “On synopses for distinct-value estimation under multiset operations,” in *Proceedings of the 2007 ACM SIGMOD International Conference on Management of Data (SIGMOD 2007)*, 2007, pp. 199–210.
- [28] D. Ting, “Towards optimal cardinality estimation of unions and intersections with sketches,” 2016, KDD 2016. [Online]. Available: <https://www.kdd.org/kdd2016/papers/files/rfp0467-tingA.pdf>
- [29] D. Ting, “Approximate distinct counts for billions of datasets,” 2019, SIGMOD 2019 / Tableau Research. [Online]. Available: <https://www.tableau.com/research/publications/approximate-distinct-counts-billions-datasets>